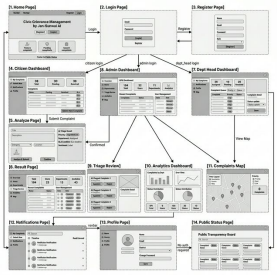
Final Year Project Report

Jan-Sunwai AI

Automated Visual Classification, Routing, and Lifecycle Tracking for Civic Grievances



Project Domain:	Civic Technology and Grievance Redressal
Technology Stack:	React, FastAPI, MongoDB, Ollama, Docker
Academic Session:	2025–2026
Project Duration:	28 January 2026 – 27 May 2026
Submission Type:	Major Project Final Report

Submitted By:	*Project Team*
Guided By:	*Project Supervisor*
Department:	*Computer Science / Information Technology*
Institute:	*Institution Name*

Contents

List of Abbreviations **ix**

List of Definitions **x**

List of Screenshots **xii**

1 Introduction **1**
1.1 Project Details . . . 1
1.2 Purpose . . . 1
1.3 Scope . . . 2
1.4 Objectives . . . 2
1.5 Technology Stack Used . . . 3
1.6 Problem Context and Motivation . . . 3
1.7 Literature Review . . . 3
1.7.1 Digital Governance and Complaint Systems . . . 3
1.7.2 Vision-Language Models in Public Workflows . . . 3
1.7.3 Local Inference and Privacy Considerations . . . 4
1.7.4 Hybrid Deterministic + AI Pipelines . . . 4
1.7.5 Governance, Explainability, and Auditability . . . 4
1.8 Chapter Summary . . . 4

2 Project Management **5**
2.1 Feasibility Study . . . 5
2.1.1 Technical Feasibility . . . 5
2.1.2 Time Schedule Feasibility . . . 5
2.1.3 Operational Feasibility . . . 5
2.1.4 Implementation Feasibility . . . 6
2.2 Project Planning . . . 6
2.2.1 Development Approach and Justification . . . 6
2.2.2 Milestones and Deliverables . . . 6
2.2.3 Roles and Responsibilities . . . 7
2.2.4 Group Dependencies . . . 8
2.3 Project Scheduling (Gantt/PERT Interpretation) . . . 8
2.3.1 High-Level Schedule . . . 8
2.3.2 PERT-Like Dependency Flow . . . 8
2.4 Risk Management . . . 8
2.5 Management Outcomes . . . 9
2.6 Detailed Week-Wise Execution Digest . . . 9
2.7 Project Governance Metrics . . . 11
2.8 Management Retrospective . . . 11
2.9 Chapter Summary . . . 11

3 System Requirement Study **12**
3.1 Existing System Overview . . . 12
3.2 Limitations of the Existing System . . . 12
3.3 User Characteristics . . . 13
3.4 Functional Requirements . . . 13
3.5 Non-Functional Requirements . . . 14
3.6 Hardware and Software Requirements . . . 15
3.6.1 Hardware Requirements . . . 15
3.6.2 Software Requirements . . . 16
3.7 Constraints . . . 16
3.7.1 UI Constraints . . . 16
3.7.2 Communication Interface . . . 16
3.7.3 Hardware Interface . . . 16
3.7.4 Criticality of Application . . . 16
3.7.5 Safety and Security Considerations . . . 16
3.8 Assumptions and Dependencies . . . 17
3.9 Detailed Use-Case Narratives . . . 17
3.10 Requirement Priority Classification . . . 18
3.11 Chapter Summary . . . 18

4 Proposed System Requirements **20**
4.1 Overview of Proposed System . . . 20
4.2 Module Descriptions . . . 20
4.2.1 Authentication and User Management Module . . . 20
4.2.2 Analyze and Complaint Intake Module . . . 20
4.2.3 Complaint Lifecycle Module . . . 20
4.2.4 Assignment and Worker Operations Module . . . 20
4.2.5 Notification and Communication Module . . . 21
4.2.6 Triage and Governance Module . . . 21
4.2.7 Analytics and Public Transparency Module . . . 21
4.3 System Features . . . 21
4.4 Advantages of the Proposed System . . . 22
4.4.1 Citizen-Centric Advantages . . . 22
4.4.2 Administrative and Departmental Advantages . . . 22
4.4.3 Institutional Advantages . . . 22
4.5 Requirement Realization Matrix . . . 22
4.6 Proposed Operational Workflow . . . 23
4.7 Expected Impact . . . 23
4.8 Module Interaction Contracts . . . 24
4.9 Requirement Realization Detail by Persona . . . 24
4.10 Proposed Data and Control Flow Narrative . . . 25
4.11 Chapter Summary . . . 25

5 System Design **26**
5.1 System Architecture Design . . . 26
5.1.1 Architectural Style . . . 26
5.1.2 Key Architectural Decisions . . . 27
5.2 UML Diagrams . . . 28
5.2.1 E-R Diagram . . . 28
5.2.2 Use Case Diagram . . . 28
5.2.3 Class Diagram . . . 29
5.2.4 Sequence Diagram . . . 29
5.2.5 Activity Diagram . . . 30
5.2.6 DFD Diagram . . . 30
5.2.7 Deployment Diagram . . . 31
5.3 Database Design . . . 31
5.3.1 Collection Design and Relationships . . . 31
5.3.2 Normalization Strategy . . . 31
5.3.3 Data Dictionary . . . 31
5.3.4 Indexing Strategy . . . 33
5.3.5 Assignment Distance Formula . . . 34
5.4 GUI Design . . . 34
5.4.1 Design Principles . . . 34
5.4.2 Interface Areas . . . 34
5.4.3 Wireframe Reference . . . 35
5.5 Screenshots . . . 35
5.5.1 Primary Screenshot Gallery (S01–S15) . . . 35
5.5.2 Supplementary Screens (Optional in Final Print) . . . 50
5.6 Chapter Summary . . . 55

6 Implementation Planning **56**
6.1 Implementation Environment . . . 56
6.1.1 Local Development Environment . . . 56
6.1.2 Production-Style Container Environment . . . 56
6.2 Tools and Technologies Used . . . 56
6.3 Coding Standards Followed . . . 57
6.3.1 Backend Standards . . . 57
6.3.2 Frontend Standards . . . 57
6.3.3 Configuration Standards . . . 57
6.4 Implementation Work Packages . . . 58
6.4.1 Work Package 1: Core Backend and Data Layer . . . 58
6.4.2 Work Package 2: AI Pipeline Integration . . . 58
6.4.3 Work Package 3: Frontend Role Flows . . . 58
6.4.4 Work Package 4: Operational Features . . . 58
6.4.5 Work Package 5: Security and Reliability Hardening . . . 58
6.4.6 Work Package 6: Deployment and Handover Readiness . . . 58
6.5 Implementation Sequence and Dependencies . . . 59

6.6 Configuration and Secret Planning . . . 59
6.7 Quality Gates in Implementation Plan . . . 60
6.8 Resource and Effort Planning . . . 60
6.9 Detailed Module Implementation Notes . . . 60
6.9.1 Backend Router Implementation Notes . . . 60
6.9.2 Service-Layer Planning Notes . . . 61
6.10 Deployment Runbook and Rollout Plan . . . 62
6.10.1 Rollout Stages . . . 62
6.10.2 Rollback Planning . . . 62
6.11 Operational Monitoring Plan . . . 62
6.12 Change Management and Release Strategy . . . 63
6.12.1 Release Checklist . . . 63
6.12.2 Implementation Learnings . . . 63
6.13 Chapter Summary . . . 63

7 Testing **64**
7.1 Testing Plan . . . 64
7.1.1 Testing Objectives . . . 64
7.1.2 Test Environment . . . 64
7.1.3 Coverage Matrix . . . 64
7.2 Types of Testing . . . 65
7.2.1 Unit Testing . . . 65
7.2.2 Integration Testing . . . 65
7.2.3 System Testing . . . 66
7.2.4 User Acceptance Testing (UAT) . . . 66
7.3 Testing Techniques . . . 66
7.3.1 Black-Box Behavioral Testing . . . 66
7.3.2 Negative Testing . . . 66
7.3.3 Resilience and Fault Injection . . . 67
7.3.4 Performance and Load Testing . . . 67
7.4 Defect Logging . . . 67
7.4.1 Defect Classification . . . 67
7.4.2 Defect Log Snapshot . . . 67
7.5 Testing Outcomes . . . 68
7.6 Residual Risks and Test Gaps . . . 68
7.7 Detailed UAT Case Matrix . . . 68
7.8 Load Benchmark Profiles and Acceptance Rubric . . . 70
7.9 Security Regression Matrix . . . 70
7.10 Chapter Summary . . . 71

8 Limitations and Future Scope **72**
8.1 Current Limitations . . . 72
8.1.1 Model and AI Limitations . . . 72
8.1.2 Infrastructure Limitations . . . 72

8.1.3 Workflow and Data Limitations . . . 72
8.1.4 Security and Governance Limitations . . . 72
8.2 Future Scope . . . 72
8.2.1 AI and Intelligence Enhancements . . . 72
8.2.2 Operational Workflow Enhancements . . . 73
8.2.3 Platform and Deployment Enhancements . . . 73
8.2.4 Security and Compliance Enhancements . . . 73
8.3 Roadmap Perspective . . . 73
8.4 Risk-Prioritized Improvement Backlog . . . 73
8.5 Adoption Considerations for Municipal Deployment . . . 75
8.5.1 Institutional Readiness Factors . . . 75
8.5.2 Change Management Considerations . . . 75
8.5.3 Data Governance Considerations . . . 75
8.6 Research Continuation Opportunities . . . 75
8.7 Chapter Summary . . . 75

9 Conclusion and References **76**
9.1 Conclusion . . . 76

10 Appendices **78**
10.1 Appendix A: API Endpoint Snapshot . . . 78
10.2 Appendix B: Environment Variable Catalog . . . 79
10.3 Appendix C: Canonical Department Taxonomy and Routing . . . 81
10.4 Appendix D: Script Inventory . . . 81
10.5 Appendix E: Screenshot Asset Checklist . . . 81
10.6 Appendix F: Sample Request and Response Payloads . . . 83
10.6.1 Login Request (Form Encoded) . . . 83
10.6.2 Analyze Request (Multipart) . . . 83
10.6.3 Analyze Response (Representative) . . . 83
10.6.4 Complaint Creation Payload (Representative) . . . 83
10.6.5 Triage Decision Payload . . . 84
10.7 Appendix G: Consolidated Execution Timeline . . . 84
10.8 Appendix H: Requirement-to-Test Traceability Matrix . . . 85

List of Figures

4.1 Proposed End-to-End Workflow 23

5.1 Figure 1: Overall System Context Diagram 26
5.2 Figure 2: Component Architecture Diagram 27
5.3 Figure 3: E-R Diagram 28
5.4 Figure 4: Use Case Diagram 28
5.5 Figure 5: Class Diagram 29
5.6 Figure 6: Sequence Diagram 29
5.7 Figure 7: Activity Diagram 30
5.8 Figure 8: Data Flow Diagram 30
5.9 Figure 9: Deployment Diagram 31
5.10 GUI Wireframe Reference from Project Documentation 35
5.11 S01: Home Page Hero and Public Entry CTA 36
5.12 S02: Login Page 37
5.13 S03: Citizen Registration Page 38
5.14 S04: Reset Password Page 39
5.15 S05: Admin Dashboard Overview 40
5.16 S06: Triage Human Review Queue 41
5.17 S07: Worker Assignment Panel 42
5.18 S08: Analytics Dashboard 43
5.19 S09: Complaints Map Overview (Satellite Layer) 44
5.20 S10: Map Popup Detail View 45
5.21 S11: Public Grievance Board 46
5.22 S12: Grievance Heatmap Page 47
5.23 S13: Profile Page (Admin) 48
5.24 S14: Citizen Dashboard with Timeline 49
5.25 S15: Notifications Dropdown Panel 50
5.26 SX1: File Complaint Upload Interface 51
5.27 SX2: Multilingual Citizen Timeline Snapshot 52
5.28 SX3: Resolved Complaint with Citizen Feedback 53
5.29 SX4: Result Page with Location Context and AI Analysis 54
5.30 SX5: Result Page with Grievance Description Editing 55

List of Tables

1.1 Project Profile 1
1.2 Objectives and Expected Outcomes 2
1.3 Technology Stack and Role in System 3

2.1 Milestone Summary and Deliverables 6
2.2 Role Responsibility Matrix (Project Execution) 7
2.3 PERT-Style Activity Dependencies 8
2.4 Risk Register and Mitigation Plan 9
2.5 Week-by-Week Execution Summary (W1–W18) 9
2.6 Governance-Oriented Delivery Metrics 11

3.1 Limitations in Legacy Grievance Handling 12
3.2 User Characteristics and Behavioral Expectations 13
3.3 Functional Requirement Specification 13
3.4 Non-Functional Requirement Specification 15
3.5 Hardware Requirement Baseline 15
3.6 Software Environment Requirements 16
3.7 Assumptions and External Dependencies 17
3.8 Narrative Use-Case Requirements 17
3.9 MoSCoW-Style Requirement Priority Mapping 18

4.1 Proposed Feature Set and Operational Benefit 21
4.2 Traceability from Requirements to Proposed Modules 22
4.3 Expected Impact of Proposed System 23
4.4 Inter-Module Contract Summary 24
4.5 Persona-Oriented Realization Snapshot 24

5.1 Major Design Decisions and Rationale 27
5.2 Data Dictionary (Core Fields) 32
5.3 Indexing Strategy by Collection 33
5.4 GUI Areas and Purpose 34

6.1 Implementation Toolchain and Use Cases 56
6.2 Implementation Sequence with Dependency Notes 59
6.3 Implementation Quality Gates 60
6.4 Router-Level Implementation Planning Notes 60
6.5 Service-Level Implementation Planning 61
6.6 Deployment Rollout Stages 62
6.7 Operational Monitoring Matrix 62

7.1 Testing Environment Baseline 64
7.2 Testing Coverage Matrix by Suite 64
7.3 Representative Unit Test Cases 65

7.4 System Test Scenarios 66
7.5 Defect Log Snapshot and Resolution 67
7.6 Outcome Summary by Validation Area 68
7.7 Persona-Driven UAT Matrix 68
7.8 Load Profile and Acceptance Guidance 70
7.9 Security Regression Verification Matrix 71

8.1 Indicative Post-Submission Roadmap 73
8.2 Future Backlog Prioritization Matrix 74

10.1 API Endpoint Snapshot 78
10.2 Core Backend Environment Variables 80
10.3 Department to Authority Mapping Snapshot 81
10.4 Operational Script Inventory 81
10.5 Screenshot Capture Checklist 82
10.6 Execution Timeline Digest for Final Report Verification 84
10.7 Traceability from Functional Requirement to Validation Evidence 85

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DB	Database
DFD	Data Flow Diagram
ER	Entity Relationship
EXIF	Exchangeable Image File Format
GPU	Graphics Processing Unit
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
NDMC	New Delhi Municipal Council
OOM	Out of Memory
OS	Operating System
PERT	Program Evaluation Review Technique
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirement Specification
TTL	Time To Live
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
XSS	Cross Site Scripting

List of Definitions

Term	Definition
Civic Complaint	A citizen-reported issue related to municipal infrastructure, utilities, sanitation, public safety, or governance services.
Complaint Lifecycle	Ordered state progression of a complaint such as Open, In Progress, Resolved, and Rejected.
Status History	Auditable log of state changes with timestamp, actor identity, and note.
Authority Routing	Mapping from classified category to responsible authority identifier and escalation parent.
Triage Queue	Review queue for low-confidence AI outputs requiring administrator decision.
Service Area	Worker-specific geospatial zone defined by latitude, longitude, and radius in kilometers.
Auto-Assignment	Rule-based process selecting an eligible worker based on department, availability, and workload.
Escalation Loop	Background process that pushes unresolved complaints to higher authorities when SLA conditions fail.
Rule Engine	Deterministic keyword and scoring logic used to improve category stability before optional LLM reasoning.
Hybrid Pipeline	Multi-stage architecture combining vision extraction, deterministic rules, and selective reasoning model invocation.
Draft Generation	AI-assisted creation of formal grievance text from image evidence and extracted metadata.
Graceful Degradation	Controlled failure behavior where the API returns meaningful retryable errors without service crash.
Magic Number Validation	File integrity check that verifies image binary headers against extension and MIME type.
Public Transparency Feed	Anonymized read-only endpoint exposing selected complaint records for public visibility.
Readiness Probe	Endpoint validating dependency availability, especially MongoDB and model runtime health.
Rate Limiting	Request throttling control to mitigate abuse and resource exhaustion.
Token Hashing	Storage of reset-token digest instead of raw token to reduce credential leakage risk.
Canonical Taxonomy	Standardized department labels used across classifier output, dashboards, and routing.

Term	Definition
Operational Governance	Administrative controls for approvals, transfers, notes, audit review, and reporting.

List of Screenshots

The report currently uses a 15-shot primary gallery and 5 supplementary slots. The table below maps identifiers, descriptions, and expected file names under `docs/images/screenshots/`.

ID	Description	Expected File Name
S01	Home page hero and call-to-action	`S01_home_desktop.png`
S02	Login page	`S02_login_desktop.png`
S03	Register page	`S03_register_desktop.png`
S04	Reset password page	`S04_reset_password_desktop.png`
S05	Admin dashboard overview	`S05_admin_dashboard_desktop.png`
S06	Triage human review table	`S06_triage_review_desktop.png`
S07	Worker assignment panel	`S07_worker_assignment_panel_desktop.png`
S08	Analytics dashboard	`S08_analytics_dashboard_desktop.png`
S09	Complaints map overview (satellite)	`S09_map_overview_satellite_desktop.png`
S10	Map popup detail view	`S10_map_popup_detail_desktop.png`
S11	Public transparency board	`S11_public_board_desktop.png`
S12	Grievance heatmap page	`S12_heatmap_desktop.png`
S13	Profile page (admin)	`S13_profile_admin_desktop.png`
S14	Citizen dashboard (complaint list/timeline)	`S14_citizen_dashboard_desktop.png`
S15	Notifications dropdown panel	`S15_notifications_panel_desktop.png`
SX1	Supplementary: file complaint upload page	`SX1_file_complaint_upload_desktop.png`
SX2	Supplementary: multilingual timeline snapshot	`SX2_citizen_timeline_multilingual_desktop.png`
SX3	Supplementary: resolved complaint with feedback	`SX3_resolved_feedback_snapshot_desktop.png`
SX4	Supplementary: result page location and AI analysis panel	`SX4_result_location_analysis_desktop.png`
SX5	Supplementary: result page grievance description editor	`SX5_result_grievance_description_desktop.png`

Expected screenshot directory: `docs/images/screenshots/`

If a file is missing during compilation, the report will render a placeholder block automatically.

Chapter 1: Introduction

Jan-Sunwai AI is an end-to-end civic grievance redressal platform built to reduce the operational gap between citizen complaint submission and departmental action. The system transforms an image upload into a structured, auditable complaint workflow, then routes the case to accountable municipal actors. Instead of requiring citizens to understand bureaucratic departments in advance, the platform performs AI-assisted classification and drafting while preserving a human-governed correction path through triage and administrative controls.

The project was developed as a local-first system to satisfy practical constraints in municipal settings where cloud deployment may be restricted by policy, infrastructure, or privacy concerns. The architecture integrates a React frontend, FastAPI backend, MongoDB persistence, and locally hosted Ollama models in a hybrid inference pipeline. This deployment posture allows the platform to run in constrained institutional environments while maintaining modularity for future expansion.

1.1 Project Details

Table 1.1: Project Profile

Attribute	**Value**
Project Name	Jan-Sunwai AI
Application Domain	Civic Grievance Management and Municipal Operations
Problem Theme	Image-based complaint intake with authority routing and lifecycle traceability
Project Duration	28 January 2026 to 27 May 2026
Primary Users	Citizens, Workers, Department Heads, Administrators
Deployment Style	Local-first, production-ready containerized stack
Target Institutional Context	NDMC-style municipal governance operations

1.2 Purpose

Traditional complaint systems rely heavily on manual typing, manual category selection, and office-level routing knowledge that many citizens do not have. This results in poor-quality submissions, delayed routing, and low trust in resolution timelines. The purpose of Jan-Sunwai AI is to build a practical, auditable, and secure system where:

- Citizens can submit evidence-driven complaints quickly using photos.
- The system auto-detects likely department categories and drafts complaint text.
- Authorities can monitor, assign, escalate, and resolve complaints with full history.
- Governance stakeholders can access analytics, public transparency views, and exports.

The objective is not to replace human decision-making but to reduce repetitive effort and routing ambiguity while strengthening accountability.

1.3 Scope

The implemented scope includes intake, classification, routing metadata, assignment, notifications, escalation support, and governance analytics. Specifically, the platform covers:

- User management with role-aware access control.
- AI analysis and complaint drafting from uploaded images.
- Complaint lifecycle actions: create, track, update, transfer, escalate, and close.
- Department-specific work queues and worker assignment controls.
- Triage process for low-confidence or ambiguous classifications.
- Notification chain and password reset mechanisms.
- Public-facing anonymized complaint listing and operational analytics.

Out-of-scope items for the current release include multilingual voice intake, direct integration with external municipal ERP systems, and predictive resource planning beyond dashboard analytics.

1.4 Objectives

Table 1.2: Objectives and Expected Outcomes

No.	Objective	Expected Outcome
O1	Reduce submission friction for citizens	Simplified upload-analyze-submit flow with optional draft editing
O2	Improve complaint routing quality	Canonical department taxonomy and authority mapping
O3	Ensure operational traceability	Status history, notes, comments, and audit-ready timeline
O4	Support workload distribution	Auto-assignment by department, service area, and load balancing
O5	Strengthen security posture	JWT auth, input sanitization, upload validation, and CORS/security headers
O6	Improve governance transparency	Analytics dashboard, heatmap, and public complaint feed
O7	Enable production transition	Docker-based deployment profile, environment controls, and runbooks

1.5 Technology Stack Used

Table 1.3: Technology Stack and Role in System

Layer	Primary Technology	Usage in Project
Frontend	React + Vite	Role-based SPA, dashboards, analyze/result flows, authentication-aware routing
Backend API	FastAPI + Python	REST endpoints, validation, business logic, authentication, and middleware
Database	MongoDB	Document storage for users, complaints, notifications, and reset tokens
AI Runtime	Ollama models	Vision extraction, optional reasoning, and complaint draft generation
Containerization	Docker Compose	Reproducible dev/prod deployment and dependency orchestration
Testing	pytest, httpx, locust	Unit/integration/security/resilience/load testing coverage
Documentation	Markdown + LaTeX	Technical runbooks, architecture notes, final submission report

1.6 Problem Context and Motivation

Civic systems face a recurring quality bottleneck at intake. Complaints often contain incomplete locations, weak descriptions, or wrong category labels. This creates downstream inefficiency where staff first reinterpret the complaint before executing field actions. In high-volume departments, this translation step alone can cause significant delay.

Jan-Sunwai AI addresses this bottleneck by introducing structured automation at the earliest stage while preserving administrative authority. The AI pipeline provides category and draft suggestions, but triage and role controls maintain governance oversight. This balance between automation and control is central to the system's design philosophy.

1.7 Literature Review

1.7.1 Digital Governance and Complaint Systems

Digital grievance platforms are a recognized mechanism for increasing civic participation and service responsiveness. However, many systems remain form-driven and text-heavy, creating barriers for users with low digital literacy or limited procedural awareness. Literature and practitioner reports emphasize that complaint quality is directly linked to downstream processing quality.

1.7.2 Vision-Language Models in Public Workflows

Vision-language models can extract useful context from unstructured images but may struggle with highly localized civic semantics. A model can correctly describe a scene yet still infer the wrong responsible department. Prior practical studies suggest that combining AI extraction with

deterministic control layers improves consistency in governance use cases.

1.7.3 Local Inference and Privacy Considerations

Public institutions frequently operate under constraints where sensitive citizen data should remain within controlled infrastructure boundaries. Local inference minimizes external data transfer and provides better control over model lifecycle, though it introduces operational concerns around VRAM budgeting, process health, and failover behavior. This project adopts local inference as a policy-aware technical choice rather than only a performance decision.

1.7.4 Hybrid Deterministic + AI Pipelines

Recent engineering patterns favor staged AI systems over monolithic generation. In this approach, computer vision extracts structured cues, deterministic rules provide stable decisions for common cases, and a lightweight reasoning model resolves ambiguity only when needed. Jan-Sunwai AI follows this pattern to reduce hallucination risk and improve repeatability.

1.7.5 Governance, Explainability, and Auditability

In civic applications, explainability is not optional. Decision provenance (confidence, method, timestamp, actor) enables triage, accountability, and policy audits. Therefore, the platform records status history and decision metadata, supports internal notes and comments, and surfaces traces through dashboards and logs.

The project design is aligned with development principles and platform guidance from FastAPI, React, MongoDB, and OWASP security best-practice references listed in Chapter 9.

1.8 Chapter Summary

This chapter defined the problem setting, project intent, scope boundaries, and technology foundation. It also established the literature-aligned rationale for adopting a hybrid AI plus deterministic architecture in a governance-sensitive municipal context. The next chapter details project planning, feasibility analysis, schedules, and delivery governance.

Chapter 2: Project Management

Project management for Jan-Sunwai AI followed an iterative delivery approach with explicit weekly milestones, acceptance checkpoints, and operational hardening gates. The planning strategy balanced feature development with risk reduction, especially in AI reliability, security posture, and deployment reproducibility.

2.1 Feasibility Study

2.1.1 Technical Feasibility

The selected stack (React, FastAPI, MongoDB, Ollama) was technically feasible for local-first civic deployment due to mature ecosystem support, strong documentation, and compatibility with available development infrastructure. Key feasibility observations were:

- FastAPI provided rapid API development, typed schemas, and OpenAPI generation.
- MongoDB supported flexible document structures suited to complaint histories, comments, and embedded metadata.
- React enabled role-specific UI composition with route guards and dashboard specialization.
- Ollama provided local model execution with configurable model tiers.
- Docker Compose enabled reproducible environment orchestration across development and production-like setups.

A major early technical risk was model reliability under constrained VRAM. This was mitigated by introducing a staged inference pipeline and avoiding simultaneous heavy model loading.

2.1.2 Time Schedule Feasibility

The schedule was structured over 18 weeks, progressing from foundation to hardening and final submission. Work cadence used Monday–Saturday windows with planned Sunday off-days and second/fourth Saturday breaks. This realistic scheduling improved sustainability while preserving delivery continuity.

2.1.3 Operational Feasibility

Operational feasibility was validated by mapping actual municipal personas into role-specific workflows:

- Citizen: complaint intake and tracking.
- Worker: assignment execution and completion updates.
- Department Head: queue supervision and status transitions.
- Admin: approvals, triage governance, analytics, and exports.

The resulting workflow matched expected governance practices where supervisory roles require both visibility and controlled override capabilities.

2.1.4 Implementation Feasibility

Implementation was feasible under both manual and scripted setup modes. Automation scripts were created for Windows and Linux, and production deployment was validated through containerized orchestration. Environment configuration was standardized via a single backend environment file to reduce drift and support repeatable handover.

2.2 Project Planning

2.2.1 Development Approach and Justification

The project used an incremental and verification-driven approach with the following cycle:

1. Build baseline functionality for a module.
2. Validate through unit and integration checks.
3. Expose role workflow in UI and test end-to-end behavior.
4. Add security/performance hardening after functional stabilization.
5. Document operational procedures and release notes.

This approach was chosen over a strict waterfall model because AI behavior and UX patterns required iterative calibration.

2.2.2 Milestones and Deliverables

Table 2.1: Milestone Summary and Deliverables

Window	Theme	Key Deliverables	Status
Weeks 1–2	Foundation	Repo setup, environment bootstrap, backend skeleton, DB connectivity, auth schema, complaint schema, initial CRUD and upload service	Completed
Weeks 3–5	AI Pipeline	Model research, classifier iterations, geotagging, reverse geocoding, generation service, analyze endpoint integration	Completed
Week 6	Frontend Base	Routing, upload flow, analyze hook, result view, integration with backend auth and analyze pipeline	Completed
Week 7	Flow Closure	Map marker improvements, local draft persistence, error boundary, 404 route, full persona walkthrough	Completed
Week 8	Security Layer	Upload hardening, rate limiting, sanitization, CORS lockdown, security headers, index hardening	Completed
Week 9	NDMC Feature Pack	Worker panel, assignment engine, audit trail, analytics/heatmap, SLA badges, bulk actions/exports	Completed

Window	Theme	Key Deliverables	Status
Week 10	Notification and Communication	Navbar badge, status-triggered notifications, email stub, forgot/reset password, profile editing, E2E notification chain test	Completed
Week 11	Performance and Reliability	Image compression, lazy loading, graceful 503 degrade, profiling, resilience tests	Completed
Week 12	Production Hardening	Docker production files, compose profile, env plan, API versioning, backup strategy, UI audit	Completed
Weeks 13–14	Testing and Pre-Handover	Unit/integration/security/load tests, mobile fixes, handover guide, API reference, repository cleanup, CI workflows	Completed
Weeks 15–16	UAT and Staging Validation	Clean deploy simulation, production verification, UAT persona runs, feedback loop and polish pass	Completed
Weeks 17–18	Reporting and Submission	Technical report artifacts, screenshots, presentation, final submission hygiene and release tagging	Completed

2.2.3 Roles and Responsibilities

Table 2.2: Role Responsibility Matrix (Project Execution)

Role Area	Responsibilities
Backend Engineering	API design, schema validation, business logic, role authorization, lifecycle transitions, notification hooks
Frontend Engineering	Route architecture, role dashboards, analyze/result flow, user session handling, responsive behavior
AI and Classification	Vision model integration, rule engine tuning, reasoning fallback, draft generation quality checks
Data and Persistence	MongoDB collection design, indexes, query access paths, reset token TTL and audit metadata
Security and Reliability	Input sanitization, upload hardening, CORS and headers, degrade behavior, readiness/liveness semantics
Testing and QA	Unit and integration tests, resilience and security validation, load profiling, UAT scripts
DevOps and Deployment	Compose consolidation, environment normalization, production verification scripts, backup runbooks
Documentation and Handover	API reference, deployment guides, user manual, architecture and report compilation

2.2.4 Group Dependencies

The project includes cross-cutting dependencies that were managed explicitly:

- Backend depends on MongoDB availability and correctly configured environment variables.
- Analyze flow depends on file storage service and Ollama runtime health.
- Frontend route behavior depends on JWT session state and role claims.
- Assignment quality depends on worker profile completeness (department, approval, service area, status).
- Production readiness depends on compose health checks, proxy alignment, and secure secret configuration.

Dependency risks were controlled through health endpoints, fallback behaviors, and scripted verification routines.

2.3 Project Scheduling (Gantt/PERT Interpretation)

2.3.1 High-Level Schedule

The schedule can be interpreted as a four-phase progression:

1. Foundation and Core Build (Weeks 1–6)
2. Security and NDMC-oriented Feature Hardening (Weeks 7–14)
3. UAT and Operational Validation (Weeks 15–16)
4. Reporting and Submission Closure (Weeks 17–18)

2.3.2 PERT-Like Dependency Flow

Table 2.3: PERT-Style Activity Dependencies

Activity ID	Activity	Depends On
A1	Core backend and DB setup	None
A2	Auth and role scaffolding	A1
A3	Analyze pipeline integration	A1, A2
A4	Frontend role flows	A2, A3
A5	Security hardening	A2, A3, A4
A6	Assignment, triage, analytics	A3, A4
A7	Notification chain and profile features	A4, A6
A8	Reliability and load profiling	A5, A6, A7
A9	Production deployment profile	A5, A8
A10	UAT and acceptance validation	A8, A9
A11	Final documentation and submission	A10

2.4 Risk Management

Table 2.4: Risk Register and Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Model unavailability or timeout	Medium	High	Multi-tier vision fallback, queue-based generation, retryable 503 responses
Wrong department classification	Medium	High	Deterministic rule engine, triage queue, admin correction workflow
Invalid/malicious uploads	Medium	High	Extension + MIME + magic number + image decode validation, max size enforcement
Unauthorized route access	Low	High	JWT auth, role-gated dependencies, integration tests for permission boundaries
Configuration drift across environments	Medium	Medium	Unified backend environment strategy and scripted deployment checks
Notification inconsistency	Low	Medium	Automated notification-chain tests and mark-all-read verification
Performance degradation under load	Medium	Medium	Profiling, timing logs, load-testing matrix, controlled fallback behavior
Documentation mismatch at handover	Medium	Medium	NDMC deployment guide, API reference sync, verification procedure chapter

2.5 Management Outcomes

The management strategy achieved three notable outcomes:

- Feature breadth was delivered without compromising security and reliability hardening.
- Delivery confidence increased through continuous testing and issue-driven iteration.
- Handover readiness improved through consolidated deployment/configuration artifacts and documentation alignment.

2.6 Detailed Week-Wise Execution Digest

Table 2.5: Week-by-Week Execution Summary (W1–W18)

Week	Date Range	Primary Execution Focus	Management Outcome
W1	Jan 28–Jan 31	Repository initialization, environment setup, architecture notes, backend bootstrap	Foundation controls established
W2	Feb 02–Feb 07	Mongo connectivity, schema definitions, early CRUD and upload service	Core API baseline available

Week	Date Range	Primary Execution Focus	Management Outcome
W3	Feb 09–Feb 14	Authentication hardening and classifier exploration	Security and AI direction validated
W4	Feb 16–Feb 21	EXIF/location extraction and reverse-geocoding integration	Analyze pipeline readiness achieved
W5	Feb 23–Feb 28	Local model runtime setup and draft generation service	AI drafting integrated
W6	Mar 02–Mar 07	Frontend routing, upload/result flow, end-to-end integration	Citizen flow stabilized
W7	Mar 09–Mar 14	Map refinement, draft persistence, error boundaries, full walkthroughs	Baseline UX hardening complete
W8	Mar 16–Mar 21	Security layer (headers, sanitizer, upload validation, limits, indexes)	Production-risk exposure reduced
W9	Mar 23–Mar 28	Assignment engine, status timeline, analytics, SLA and bulk controls	Operational depth achieved
W10	Mar 30–Apr 04	Notification chain, email stub, password reset, profile update	Communication workflows closed
W11	Apr 06–Apr 11	Performance optimization, graceful degrade, profiling, resilience tests	Reliability confidence improved
W12	Apr 13–Apr 18	Production container profile and deployment artifact maturation	Deployment readiness strengthened
W13	Apr 20–Apr 25	Test expansion (unit/integration/security/load) and mobile fixes	Quality gates expanded
W14	Apr 27–Apr 30	API/docs finalization, cleanup, CI/release prep	Pre-handover alignment completed
W15	May 04–May 09	Production verification and clean-machine deployment simulation	Reproducibility confirmed
W16	May 11–May 16	UAT persona runs, feedback loop, resilience validation	User-level acceptance confidence gained
W17	May 18–May 23	Reporting artifacts, screenshots, user manual, presentation draft	Submission assets consolidated

Week	Date Range	Primary Execution Focus	Management Outcome
W18	May 25–May 27	Final release hygiene and submission closure	Project lifecycle completed

2.7 Project Governance Metrics

Table 2.6: Governance-Oriented Delivery Metrics

Metric	Interpretation in This Project
Milestone completion consistency	Planned milestone themes were completed in sequence with controlled scope shifts and documented acceleration notes
Defect turnaround behavior	High-severity defects were fixed with targeted regression verification in the same development cycle
Quality gate maturity	Testing moved from smoke-only to layered validation (security, resilience, load, and UAT)
Deployment reproducibility	Scripts and compose profile reduced dependency on manual environment interpretation
Documentation synchronization	Technical docs, runbooks, and report artifacts were updated as implementation evolved
Operational risk closure rate	Key risk classes (auth, upload security, route drift, and notification consistency) were explicitly addressed

2.8 Management Retrospective

Three management lessons were central to delivery quality:

1. **Early role modeling reduced rework:** Defining citizen/worker/dept-head/admin pathways early prevented significant UI and authorization redesign later.
2. **Security and reliability should not be deferred:** Integrating these streams before final release avoided unstable late-stage merges.
3. **Artifact consolidation is a management task:** Unifying environment and compose artifacts reduced support burden and improved handover confidence.

2.9 Chapter Summary

This chapter demonstrated that the project was not only technically implementable but also deliverable within a controlled timeline and governance framework. Feasibility, dependency management, risk planning, and milestone tracking enabled predictable progress toward a production-oriented civic platform.

Chapter 3: System Requirement Study

This chapter documents requirement analysis for Jan-Sunwai AI, including the baseline state of existing grievance systems, observed limitations, actor expectations, and formalized functional/non-functional requirements. The study was derived from implementation realities, municipal workflow assumptions, and validation through tests and user journeys.

3.1 Existing System Overview

Many conventional civic complaint channels depend on one or more of the following interfaces: physical counters, basic web forms, call center logging, or generic ticketing portals. While these methods can register complaints, they often suffer from weak input quality, manual routing dependency, and low traceability.

Key characteristics of existing systems observed in this problem context:

- Citizens are expected to manually choose departments, often without sufficient guidance.
- Complaint narratives are text-heavy and unstructured, leading to inconsistent detail quality.
- Photo evidence may be accepted but is rarely integrated into routing logic.
- Status updates are frequently sparse or delayed.
- Workflow metadata (who changed what and why) is often incomplete for audits.

3.2 Limitations of the Existing System

Table 3.1: Limitations in Legacy Grievance Handling

ID	Observed Limitation	Operational Effect
L1	Manual authority selection by citizens	High probability of misrouting and delayed first response
L2	Inconsistent complaint description quality	Staff time spent rewriting or clarifying cases
L3	Weak integration of visual evidence	Evidence underused in initial triage and prioritization
L4	Limited role-specific workflows	Same interface burden across very different operational personas
L5	Minimal lifecycle traceability	Difficult root-cause analysis and accountability review
L6	Low transparency in queue behavior	Citizens lose trust when progress signals are unclear
L7	Sparse analytics for administrators	Planning decisions made without stable trend and workload data

3.3 User Characteristics

The system is designed for four operational personas with distinct interaction patterns and authorization boundaries.

Table 3.2: User Characteristics and Behavioral Expectations

User Role	Primary Goals	Typical Interaction Pattern
Citizen	Report issues and track outcomes	Upload image, review generated draft, submit complaint, monitor notifications and status timeline
Worker	Resolve assigned field tasks	View own queue, update availability, mark task completion, add resolution note
Department Head	Manage departmental flow	Filter department queue, change status, add notes, transfer/escalate cases when needed
Administrator	Governance and oversight	Approve workers, triage ambiguous cases, perform bulk actions, export reports, monitor analytics

3.4 Functional Requirements

Table 3.3: Functional Requirement Specification

FR ID	Requirement Name	Requirement Description
FR-01	User Registration	System shall allow self-registration for citizen and worker roles with validation and duplicate checks.
FR-02	User Authentication	System shall issue JWT access tokens on successful login and protect authorized routes.
FR-03	Profile Management	Authenticated users shall update selected profile fields such as full name and phone number.
FR-04	Password Recovery	System shall support forgot-password and reset-password flows with time-bound tokens.
FR-05	Image Upload Validation	Analyze endpoint shall accept valid JPG/PNG images and reject unsupported or malformed payloads.
FR-06	AI-Assisted Analysis	System shall analyze uploaded image and return category, confidence, location metadata, and draft status.
FR-07	Draft Generation	System shall produce a complaint draft synchronously or via queued generation, with polling support.

FR ID	Requirement Name	Requirement Description
FR-08	Complaint Creation	System shall create a complaint record from reviewed analysis payload and user-provided description edits.
FR-09	Complaint Listing	System shall provide role-filtered complaint lists with pagination and optional status filters.
FR-10	Complaint Detail View	System shall provide complete complaint detail including status history and operational metadata.
FR-11	Status Update Workflow	Department head/admin shall update complaint status with note and trigger citizen notification.
FR-12	Department Transfer	Department head/admin shall transfer complaint ownership across departments when misrouted.
FR-13	Escalation	Department head/admin shall escalate complaints to parent authorities when required.
FR-14	Worker Assignment	System shall auto-assign eligible workers and support manual override by admin.
FR-15	Worker Operations	Worker shall manage availability and mark assigned complaints as done.
FR-16	Notification Center	System shall provide list, unread count, single-read, and mark-all-read operations.
FR-17	Triage Queue	Admin shall review low-confidence complaints and approve/reject/correct labels.
FR-18	Analytics	Admin/dept-head shall access overview and geospatial heatmap summaries.
FR-19	CSV Export	Admin shall export complaint data to CSV for external reporting.
FR-20	Public Transparency Feed	System shall expose anonymized complaint records without authentication.
FR-21	Health Monitoring	System shall provide live, ready, model, and GPU health endpoints.
FR-22	API Version Alias	System shall expose major endpoints under /api/v1 for integration stability.

3.5 Non-Functional Requirements

Table 3.4: Non-Functional Requirement Specification

NFR ID	Quality Attribute	Requirement Description
NFR-01	Security	JWT auth, RBAC, sanitization, secure headers, and account-safe reset flow shall be enforced.
NFR-02	Reliability	API shall degrade gracefully under model or DB faults with explicit status semantics.
NFR-03	Performance	Public and health endpoints shall respond consistently under mixed read-heavy load.
NFR-04	Usability	Role-specific UI shall reduce cognitive load and support responsive layouts for desktop/mobile usage.
NFR-05	Maintainability	Modular routers/services with documented runbooks shall support iterative enhancement.
NFR-06	Observability	Process-time headers, rotating logs, and health routes shall support debugging and operations.
NFR-07	Scalability Readiness	Queue-based generation and container deployment shall support controlled scaling.
NFR-08	Portability	Project setup shall support Windows and Linux via scripts and Docker-based workflow.
NFR-09	Compliance Readiness	Local-first inference and sanitized logs shall support stricter governance deployments.
NFR-10	Data Integrity	Validation schemas and controlled update flows shall minimize malformed or inconsistent records.

3.6 Hardware and Software Requirements

3.6.1 Hardware Requirements

Table 3.5: Hardware Requirement Baseline

Component	Minimum	Recommended
CPU	4-core modern processor	6 to 8-core processor
System RAM	12 GB available	16 GB or higher
GPU VRAM	4 GB	6 GB or higher
Storage	10 GB free	20 GB free
Network	Stable LAN/internet for maps and package fetch	Managed institutional network

3.6.2 Software Requirements

Table 3.6: Software Environment Requirements

Software Component	Baseline	Purpose
Operating System	Windows 10+/Linux	Development and deployment host
Python Runtime	Python 3.10+	Backend and automation scripts
Node.js and npm	Current LTS range	Frontend build and tooling
MongoDB	7.x class deployment	Document persistence
Ollama Runtime	Local model runtime	Vision/reasoning generation pipeline
Docker Engine/Compose	Docker 24+ recommended	Containerized orchestration
Browser	Chromium/Firefox class	Frontend access and UAT execution

3.7 Constraints

3.7.1 UI Constraints

- Role-specific pages must enforce access boundaries and clear redirection behavior.
- Analyze/result screens must preserve draft continuity during refresh and error conditions.
- Dashboard cards and maps must remain usable at mobile widths (e.g., 375 px baseline tests).

3.7.2 Communication Interface

- REST over HTTP(S) is the canonical communication model.
- JWT bearer token is mandatory for protected routes.
- Production frontend proxy must route API traffic to backend service with /api/v1 compatibility.

3.7.3 Hardware Interface

- AI response times depend on model runtime health and available VRAM.
- GPU contention and model load/unload latency can affect throughput.
- Large image uploads require strict file-size and decode constraints to avoid resource exhaustion.

3.7.4 Criticality of Application

The system is governance-sensitive. Incorrect status transitions, misrouted complaints, or unauthorized updates can directly impact citizen trust and administrative accountability. Hence, status history and role checks are non-negotiable controls.

3.7.5 Safety and Security Considerations

- Input sanitization is required for free-text fields.

- File intake must detect extension-header mismatches and corrupted payloads.
- Token reset flow must avoid account enumeration and raw token persistence.
- Security headers and CORS policies must remain active on both normal and error responses.

3.8 Assumptions and Dependencies

Table 3.7: Assumptions and External Dependencies

ID	Assumption/Dependency	Implication
A1	MongoDB instance is reachable and healthy	Core read/write workflows remain functional
A2	Ollama runtime is available with configured models	Analyze and generation pathways remain active
A3	Environment variables are configured accurately	Auth, DB, CORS, and model endpoints behave as intended
A4	Frontend and backend URL mapping is consistent	API calls and route protection remain stable
A5	Worker profiles include valid department and status metadata	Assignment service can select candidates correctly
A6	Deployment host supports Docker where container mode is used	Production-style stack can be reproduced

3.9 Detailed Use-Case Narratives

Table 3.8: Narrative Use-Case Requirements

UC ID	Actor	Main Flow	Alternate/Exception Flow
UC-01	Citizen	Register account, login, access dashboard	Duplicate username/email is rejected with validation message
UC-02	Citizen	Upload image and run analyze pipeline	Unsupported/corrupt image rejected with explicit reason
UC-03	Citizen	Edit draft and submit complaint	If analyze unavailable, user submits manual complaint text after retry guidance
UC-04	Citizen	Track status and receive notifications	If no notifications available, panel remains stable with unread count zero
UC-05	Citizen	Submit feedback after complaint resolution	Duplicate feedback attempt blocked with conflict response

UC ID	Actor	Main Flow	Alternate/Exception Flow
UC-06	Worker	View assigned complaints and update availability	Worker pending approval cannot access worker-only operations
UC-07	Worker	Mark complaint done	If assignment mismatch occurs, operation is denied by role/ownership checks
UC-08	Dept Head	Filter department complaints and update status	Dept-head cannot update complaints outside own department
UC-09	Dept Head	Add notes and transfer/escalate complaint	Invalid complaint ID or missing target yields controlled validation error
UC-10	Admin	Approve/reject worker registrations	Already-approved/rejected states handled idempotently
UC-11	Admin	Operate triage queue and analytics	Empty triage queue still returns successful response
UC-12	Public User	View anonymized complaint board	No authenticated personal fields are exposed in response payload

3.10 Requirement Priority Classification

Table 3.9: MoSCoW-Style Requirement Priority Mapping

Priority	Requirement IDs	Interpretation
Must Have	FR-01 to FR-12, FR-14, FR-16, FR-21	Core operational viability: authentication, complaint lifecycle, role control, assignment, notifications, and service health
Should Have	FR-13, FR-15, FR-17, FR-18, FR-22	Strong governance and maintainability improvements that meaningfully increase institutional readiness
Could Have	FR-19, FR-20	Non-blocking but high-value transparency and reporting improvements
Non-Functional Critical	NFR-01, NFR-02, NFR-06, NFR-10	Security, reliability, observability, and data integrity controls required for trusted operation

3.11 Chapter Summary

The requirement study establishes the transition from a manually intensive grievance flow to a structured, secure, and auditable digital workflow. Functional and non-functional requirements

were formalized in implementation-aligned terms and mapped against realistic user roles and operational constraints.

Chapter 4: Proposed System Requirements

This chapter describes the proposed Jan-Sunwai AI solution architecture from a requirement realization perspective. While Chapter 3 documented what is required, this chapter explains how the proposed system addresses those requirements through modular capabilities and integrated workflows.

4.1 Overview of Proposed System

The proposed system is a role-driven civic complaint platform that combines AI-assisted intake with deterministic governance controls. The citizen-facing side is optimized for minimal friction, while administrative and departmental flows emphasize accountability and operational throughput.

The system can be summarized as a five-stage path:

1. Complaint Intake: Citizen uploads image and requests analysis.
2. AI Enrichment: System extracts likely category, location hints, and draft text.
3. Structured Submission: Citizen reviews/edits and confirms complaint payload.
4. Operational Routing: Complaint is assigned, tracked, and escalated if needed.
5. Governance and Transparency: Notifications, analytics, exports, and public feed support accountability.

4.2 Module Descriptions

4.2.1 Authentication and User Management Module

This module handles registration, login, session identity, profile updates, and password recovery. It enforces role-oriented authorization for citizen, worker, department head, and admin personas. Key controls include JWT token issuance, role checks, and reset-token hashing.

4.2.2 Analyze and Complaint Intake Module

This module processes uploaded images, invokes the hybrid classifier pipeline, extracts location metadata, and orchestrates draft generation. It supports both immediate and queued generation paths and returns timing metadata for observability.

4.2.3 Complaint Lifecycle Module

This module persists complaints and manages state transitions, including status updates, transfer, escalation, feedback, notes, and comments. It appends status history entries for each transition to preserve decision traceability.

4.2.4 Assignment and Worker Operations Module

This module auto-assigns complaints to eligible workers using department matching, optional geospatial service-area checks, and load balancing by active tasks. It also provides worker status controls and completion signaling.

4.2.5 Notification and Communication Module

This module creates in-app notifications for complaint lifecycle events and maintains unread counters. It includes an email relay stub for deployment environments where SMTP integration is required.

4.2.6 Triage and Governance Module

This module supports admin review of low-confidence classifications. It enables decision actions such as approve, reject, or relabel. The triage path acts as a governance safety net against AI uncertainty.

4.2.7 Analytics and Public Transparency Module

This module aggregates operational statistics, trend lines, and geospatial heatmap data. It also exposes an anonymized public feed to support transparency without leaking sensitive user data.

4.3 System Features

Table 4.1: Proposed Feature Set and Operational Benefit

Feature ID	Feature	Operational Benefit
PF-01	Image-first intake	Reduces textual burden on citizen and improves evidence quality
PF-02	Hybrid category detection	Improves classification consistency through rules plus selective reasoning
PF-03	Draft complaint generation	Standardizes language quality and reduces submission ambiguity
PF-04	Role-based dashboard routing	Ensures each persona sees relevant actions and data
PF-05	Auto-assignment service	Reduces manual workload and accelerates field execution
PF-06	Status history timeline	Improves accountability and supports audit and grievance review
PF-07	Notification chain	Keeps citizens informed about progress without repeated manual follow-up
PF-08	Triage queue	Prevents low-confidence AI outputs from silently propagating
PF-09	Bulk admin actions	Improves throughput for backlog management in peak periods
PF-10	Heatmap and analytics	Supports evidence-driven supervision and resource planning
PF-11	Public complaint feed	Increases transparency and trust in civic workflow visibility

Feature ID	Feature	Operational Benefit
PF-12	Health endpoints and timing logs	Supports operational diagnosis and production monitoring

4.4 Advantages of the Proposed System

4.4.1 Citizen-Centric Advantages

- Simplified complaint filing with image-based assistance.
- Better complaint quality due to generated formal drafts.
- Real-time feedback through status and notification channels.
- Reduced confusion in choosing the correct municipal department.

4.4.2 Administrative and Departmental Advantages

- Better routing quality and lower manual triage burden for standard cases.
- Strong lifecycle auditability for supervisory and escalation actions.
- Faster assignment and backlog control through automation and bulk operations.
- Structured data support for reporting and policy-level analysis.

4.4.3 Institutional Advantages

- Local-first inference supports stricter data residency requirements.
- Containerized deployment supports repeatable setup and handover.
- Explicit API version aliasing improves integration stability.
- Security hardening and validation controls reduce common exploit surfaces.

4.5 Requirement Realization Matrix

Table 4.2: Traceability from Requirements to Proposed Modules

Requirement Group	Primary Module	Realization Mechanism
Registration and Login	Auth and User Management	Register/login routes, JWT issuance, profile update API, password reset token lifecycle
AI Classification and Drafting	Analyze Module + LLM Queue	Vision model cascade, rule engine scoring, optional reasoning, queued generation polling
Lifecycle and Tracking	Complaint Lifecycle Module	CRUD plus status, transfer, escalate, feedback, notes, comments, status history
Worker Operations	Assignment and Worker Module	Eligibility filter, geospatial check, load balancing, worker completion flow
Governance and Review	Triage and Admin Controls	Low-confidence review queue and corrective decision actions

Requirement Group	Primary Module	Realization Mechanism
Communication	Notifications Module	In-app notification events, unread counter, mark-read APIs, email stub integration
Reporting and Transparency	Analytics/Public Module	Overview metrics, heatmap endpoint, anonymized public complaint feed
Resilience and Security	Cross-cutting Platform Controls	Health probes, security headers, upload hardening, sanitization, rate limits, logs

4.6 Proposed Operational Workflow

Proposed End-to-End Workflow

Citizen submits evidence, AI enriches payload, complaint is stored and routed, worker and department users execute actions, and governance views aggregate operational signals.

Figure 4.1: Proposed End-to-End Workflow

4.7 Expected Impact

Table 4.3: Expected Impact of Proposed System

Impact Area	Expected Outcome
Complaint quality	Higher consistency through guided input and generated drafts
Routing efficiency	Reduced first-hop delays due to department prediction and authority mapping
Resolution governance	Better oversight through status timelines, notes, and escalation controls
Citizen trust	Improved perception through notifications and transparent progress signals
Operational analytics	Better planning through department trends and heatmap indicators
Deployment readiness	Lower setup friction due to unified compose and environment strategy

4.8 Module Interaction Contracts

Table 4.4: Inter-Module Contract Summary

Producer Module	Consumer Module	Interaction Contract
Authentication module	All protected routers	Provides current user identity and role claims via dependency injection
Analyze module	Complaint lifecycle module	Produces classification/location/draft payload used for complaint creation
Complaint lifecycle module	Notification module	Emits status/registration/escalation events for user-facing notifications
Complaint lifecycle module	Assignment module	Triggers auto-assignment at complaint creation and slot-free reassignments
Worker module	Complaint lifecycle module	Reports completion updates that influence lifecycle and status history
Triage module	Complaint lifecycle module	Applies corrected decisions for low-confidence complaint records
Analytics module	Complaint data store	Executes aggregate queries for dashboard and heatmap responses
Public module	Complaint data store	Exposes anonymized subset for transparency without privileged fields
Health module	Database/model runtime	Publishes liveness/readiness/dependency status for operations

4.9 Requirement Realization Detail by Persona

Table 4.5: Persona-Oriented Realization Snapshot

Persona	Key Features Delivered	Operational Value Delivered
Citizen	Analyze intake, dashboard tracking, notifications, feedback, password reset	Reduced filing friction and improved visibility of complaint progress
Worker	Assignment list, status updates, completion actions	Structured field execution and improved queue throughput
Department Head	Department filtering, status updates, transfer/escalation, notes	Controlled supervisory governance with auditable transitions
Admin	Worker approvals, triage decisions, bulk operations, analytics, exports	End-to-end operational governance and reporting capability
Public Viewer	Public board access	Transparency without exposing personally identifiable records

4.10 Proposed Data and Control Flow Narrative

The proposed flow intentionally separates decision support from governance authority. AI modules suggest categories and draft text, but acceptance and lifecycle transitions remain controlled through authenticated role actions. This hybrid control plane avoids over-automation risk while still reducing manual burden in high-volume intake scenarios.

4.11 Chapter Summary

The proposed system translates requirement intent into integrated modules that jointly address user friction, operational traceability, and governance accountability. It deliberately combines AI assistance with deterministic controls and administrative safety nets, making it suitable for practical municipal deployment contexts.

Chapter 5: System Design

System design in Jan-Sunwai AI focuses on reliable complaint orchestration in a governance-sensitive environment. The design balances modular API responsibilities, AI integration boundaries, role-specific UI behavior, and document-oriented persistence strategies.

5.1 System Architecture Design

5.1.1 Architectural Style

The platform follows a layered service architecture:

- Presentation Layer: React SPA with role-aware route protection.
- API Layer: FastAPI routers grouped by domain concern.
- Service Layer: Assignment, queueing, sanitization, and utility services.
- Data Layer: MongoDB collections and indexes.
- AI Runtime Layer: Ollama-hosted models with staged invocation.

Figure 1: Overall System Context Diagram

Users interact with the frontend, which communicates with backend APIs. Backend coordinates MongoDB, uploads storage, AI runtime, notification logic, and escalation loop.

Figure 5.1: Figure 1: Overall System Context Diagram

Figure 2: Component Architecture Diagram

Frontend modules map to backend routers and services. Persistence collections and model runtime are isolated behind service interfaces for maintainability and testability.

Figure 5.2: Figure 2: Component Architecture Diagram

5.1.2 Key Architectural Decisions

Decision	Rationale
Hybrid AI pipeline (vision + rules + optional reasoning)	Reduces hallucination risk, lowers VRAM pressure, and improves deterministic routing for common civic scenes
Document model in MongoDB	Supports rich complaint lifecycle payloads and embedded audit metadata without heavy relational join overhead
Role-specific route gates	Enforces operational boundaries between citizen, worker, department head, and admin personas
Queue-based draft generation fallback	Prevents long-running model calls from blocking frontend experience
Security middleware and sanitization by default	Reduces exploit surface for API consumers and file uploads
Versioned API alias (/api/v1)	Improves long-term integration stability for external municipal consumers

Table 5.1: Major Design Decisions and Rationale

5.2 UML Diagrams

5.2.1 E-R Diagram

Figure 3: E-R Diagram

Primary entities: User, Complaint, Notification, and PasswordReset token. Relationships include complaint ownership, assignment, and notification generation.

Figure 5.3: Figure 3: E-R Diagram

5.2.2 Use Case Diagram

Figure 4: Use Case Diagram

Citizen submits and tracks complaints; worker resolves assignments; department head updates status; admin governs approvals, triage, and analytics.

Figure 5.4: Figure 4: Use Case Diagram

5.2.3 Class Diagram

Figure 5: Class Diagram

Schema-level classes represent User, Complaint, and Notification with association to role, status, and metadata properties.

Figure 5.5: Figure 5: Class Diagram

5.2.4 Sequence Diagram

Figure 6: Sequence Diagram

Upload -> analyze -> generate draft -> submit complaint -> auto-assign -> status updates -> notifications.

Figure 5.6: Figure 6: Sequence Diagram

5.2.5 Activity Diagram

Figure 7: Activity Diagram

Lifecycle from citizen intake through operational processing and closure with feedback and audit logging.

Figure 5.7: Figure 7: Activity Diagram

5.2.6 DFD Diagram

Figure 8: Data Flow Diagram

Data moves from frontend to API services, then to persistence and AI runtime. Notification and analytics streams derive from lifecycle events.

Figure 5.8: Figure 8: Data Flow Diagram

5.2.7 Deployment Diagram

Figure 9: Deployment Diagram

Browser clients access frontend service; frontend proxies API calls to backend service; backend communicates with MongoDB and host-level Ollama runtime.

Figure 5.9: Figure 9: Deployment Diagram

5.3 Database Design

5.3.1 Collection Design and Relationships

The database is organized around four principal collections:

- **users**: identity, role, and worker service-area metadata.
- **complaints**: complaint payload, routing metadata, status history, and collaboration artifacts.
- **notifications**: user-scoped notification events and read-state tracking.
- **password_resets**: token hash lifecycle with expiry and usage state.

The design intentionally keeps lifecycle and audit structures close to complaint documents for locality and retrieval simplicity.

5.3.2 Normalization Strategy

The platform uses controlled denormalization:

- Embedded `status_history`, `dept_notes`, and `comments` reduce query complexity.
- Core identity references (e.g., `user_id`, `assigned_to`) remain external for role and profile updates.
- Routing metadata is persisted with complaint records to preserve deterministic decision context over time.

This approach preserves operational read performance while retaining clear ownership semantics.

5.3.3 Data Dictionary

Table 5.2: Data Dictionary (Core Fields)

Collection	Field	Type	Purpose
users	_id	ObjectId	Primary identifier for user record
users	username	String	Unique login identity
users	email	String	Unique contact and recovery identity
users	password	String (hash)	bcrypt password hash
users	role	Enum/String	Citizen, worker, department head, admin
users	department	String	Department linkage for worker and head roles
users	is_approved	Boolean	Worker approval gate
users	worker_status	String	Availability state (available/offline/busy)
users	active_complaint_ids	Array[String]	Assigned active complaint references
users	service_area	Object	Worker geofence with lat, lon, radius_km
users	created_at / updated_at	Datetime	Audit timestamps
complaints	_id	ObjectId	Primary complaint identity
complaints	user_id	String	Citizen owner reference
complaints	assigned_to	String/Null	Worker currently assigned
complaints	authority_id	String	Current authority mapping
complaints	department	String	Canonical department label
complaints	status	Enum/String	Open/In Progress/Resolved/Rejected
complaints	priority	Enum/String	Derived urgency priority
complaints	description	String	Citizen complaint text
complaints	image_url	String	Stored upload path
complaints	location	Object	Geospatial metadata and address
complaints	ai_metadata	Object	Model, confidence, labels, rationale fields
complaints	routing_confidence	Float	Routing confidence metadata
complaints	escalated	Boolean	Escalation state
complaints	`escalation_parent_authority_id`	String/Null	Parent target for escalation
complaints	status_history	Array[Object]	Lifecycle transition audit
complaints	dept_notes	Array[Object]	Internal notes for admin/head
complaints	comments	Array[Object]	Shared comment thread
complaints	feedback	Object/Null	Citizen post-resolution feedback
complaints	is_duplicate / duplicate_of	Boolean/String	Duplicate detection metadata
complaints	created_at / updated_at	Datetime	Lifecycle timestamps
notifications	_id	ObjectId	Notification identifier
notifications	user_id	String	Recipient user identity

Collection	Field	Type	Purpose
notifications	complaint_id	String/Null	Related complaint reference
notifications	type	String	Notification category (status/escalation/etc.)
notifications	title	String	Display title
notifications	message	String	Display message
notifications	is_read	Boolean	Read-state for badge computation
notifications	created_at	Datetime	Event timestamp
`password_resets`	_id	ObjectId	Reset record identity
`password_resets`	user_id	String	Target user identity
`password_resets`	token_hash	String	SHA-256 hash of reset token
`password_resets`	used	Boolean	Token consumed/revoked marker
`password_resets`	created_at	Datetime	Token issued time
`password_resets`	expires_at	Datetime	TTL-controlled expiry time
`password_resets`	used_at / revoked_at / expired_at	Datetime/Null	Token lifecycle audit fields

5.3.4 Indexing Strategy

Collection	Index	Purpose
users	unique(username), unique(email)	Fast auth lookups and uniqueness enforcement
complaints	(user_id, created_at)	Efficient citizen dashboard history retrieval
complaints	(status, created_at)	Queue and status-oriented admin filtering
complaints	(department, created_at)	Department-level operational listing
complaints	authority_id	Escalation and authority review queries
notifications	(user_id, created_at)	Ordered notification feed retrieval
notifications	(user_id, is_read)	Unread badge counter queries
`password_resets`	`unique(token_hash)`	Token lookup and collision prevention
`password_resets`	`(user_id,used)`	Active token invalidation management
`password_resets`	`TTL(expires_at)`	Automatic expired-token cleanup

Table 5.3: Indexing Strategy by Collection

5.3.5 Assignment Distance Formula

Auto-assignment uses geospatial proximity filtering where service-area radii are configured for workers. The distance calculation follows the Haversine relation described by Sinnott (1984):

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right) \tag{5.1}$$

Equation (5.1) is used for worker-to-complaint distance scoring, where R is Earth radius in kilometers, (ϕ_1, λ_1) is worker location, and (ϕ_2, λ_2) is complaint location.

5.4 GUI Design

5.4.1 Design Principles

The UI follows practical governance portal principles:

- Clarity of action over decorative complexity.
- Explicit role boundaries in navigation and controls.
- Immediate visual feedback for status and notification changes.
- Responsive behavior for desktop operator screens and mobile field usage.

5.4.2 Interface Areas

Interface Area	**Purpose**
Public Home	Awareness, role entry points, and transparency access
Authentication Screens	Login, registration, forgot password, reset password
Analyze and Result Views	Upload, AI analysis display, map interaction, draft review
Citizen Dashboard	Complaint list, timeline, feedback, and progress visibility
Worker Dashboard	Assigned queue and completion updates
Department Head Dashboard	Department-level filtering, status updates, and notes
Admin Dashboard	Global queue, approvals, triage, bulk actions, analytics, exports
Notification Panel	Event feed and read-state management
Profile Screen	Personal information update controls

Table 5.4: GUI Areas and Purpose

5.4.3 Wireframe Reference

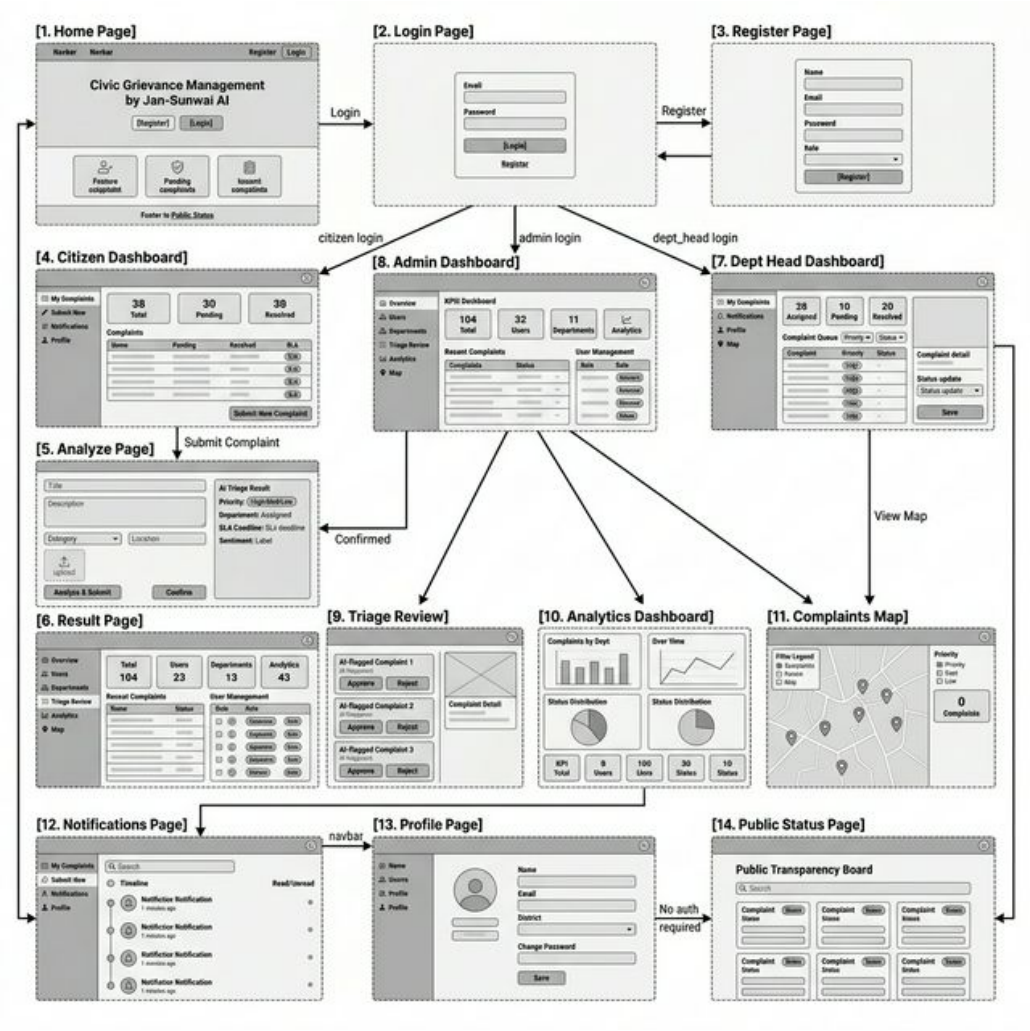


Figure 5.10: GUI Wireframe Reference from Project Documentation

5.5 Screenshots

The screenshot gallery below is designed for final submission formatting. Each figure points to a deterministic file name under `docs/images/screenshots/`. Missing files render as placeholders so report compilation can continue while pending captures are added.

5.5.1 Primary Screenshot Gallery (S01–S15)

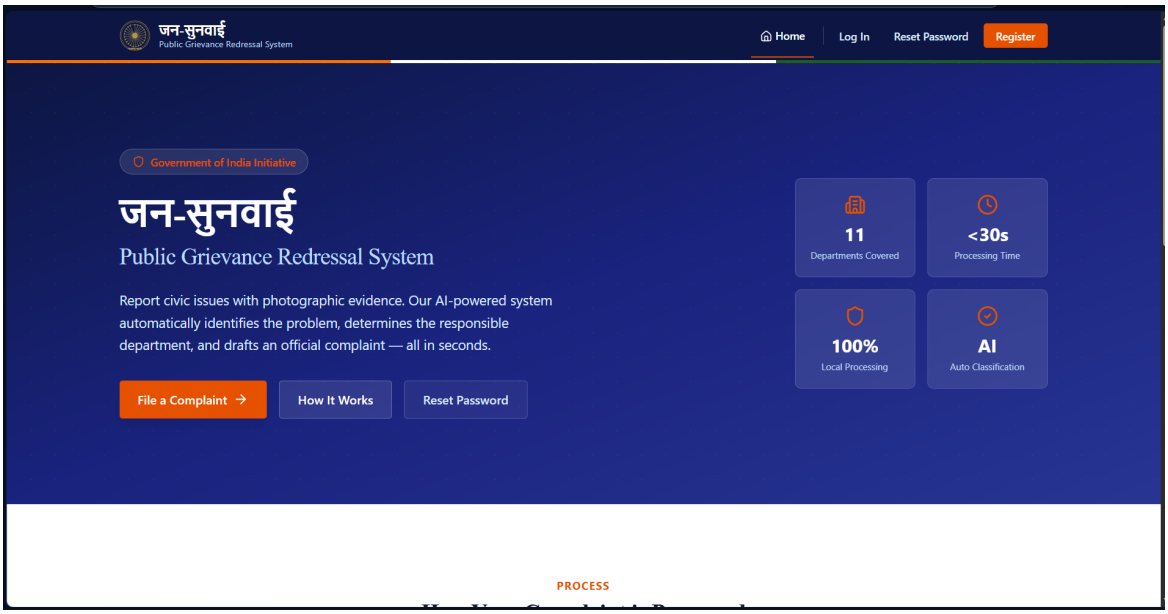


Figure 5.11: S01: Home Page Hero and Public Entry CTA

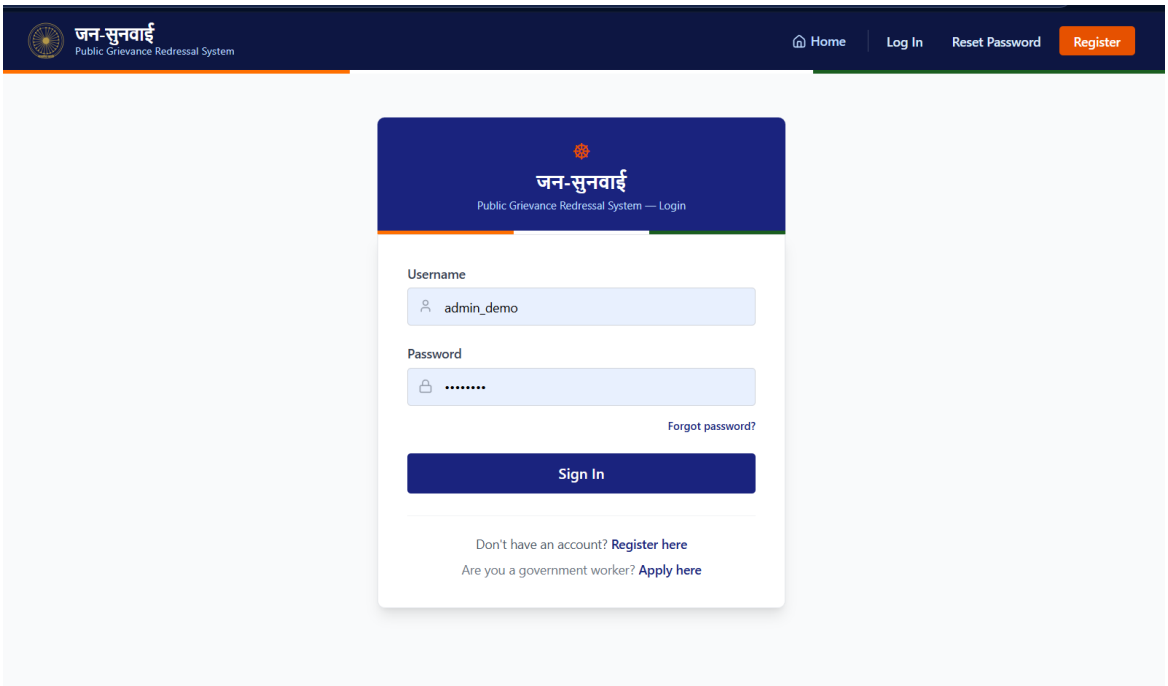


Figure 5.12: S02: Login Page

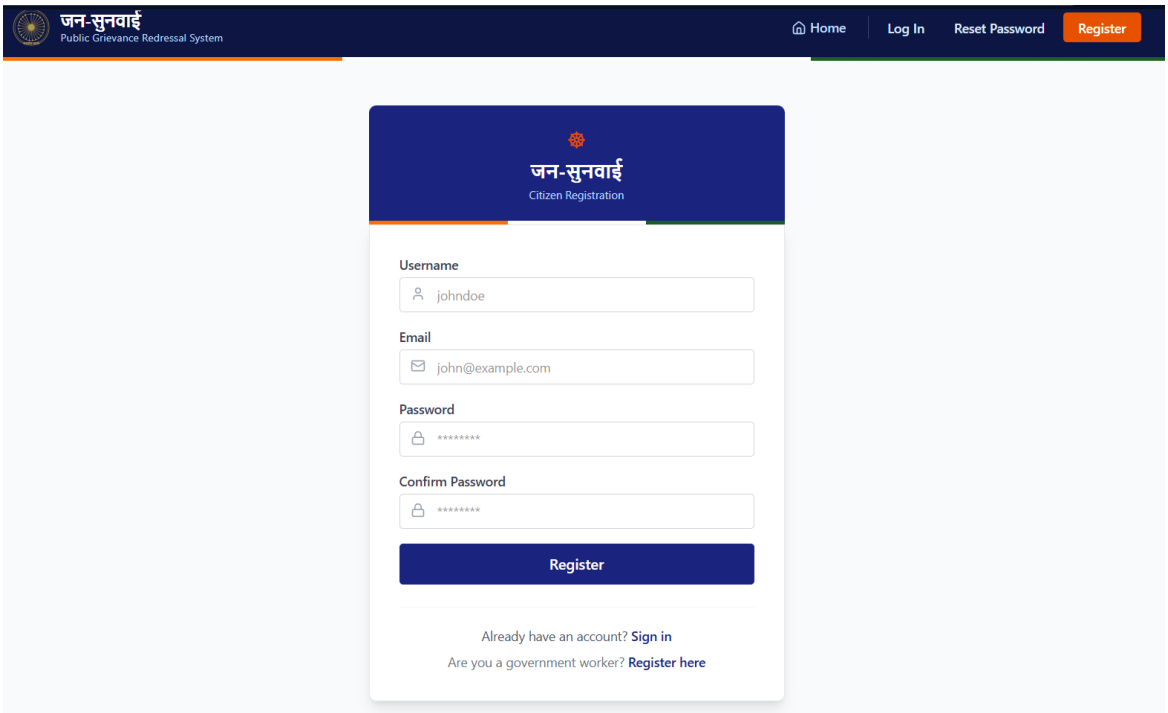


Figure 5.13: S03: Citizen Registration Page

Screenshot Placeholder

Missing file: `../../images/screenshots/S04_reset_password_desktop.png`

Figure 5.14: S04: Reset Password Page

Screenshot Placeholder

Missing file: `../../images/screenshots/S05_admin_dashboard_desktop.png`

Figure 5.15: S05: Admin Dashboard Overview

Screenshot Placeholder

Missing file: `../../images/screenshots/S06_triage_review_desktop.png`

Figure 5.16: S06: Triage Human Review Queue

Screenshot Placeholder

Missing file:
`../../images/screenshots/S07_worker_assignment_panel_desktop.png`

Figure 5.17: S07: Worker Assignment Panel

Screenshot Placeholder

Missing file: `../../images/screenshots/S08_analytics_dashboard_desktop.png`

Figure 5.18: S08: Analytics Dashboard

Screenshot Placeholder

Missing file:
`../../images/screenshots/S09_map_overview_satellite_desktop.png`

Figure 5.19: S09: Complaints Map Overview (Satellite Layer)

Screenshot Placeholder

Missing file: `../../images/screenshots/S10_map_popup_detail_desktop.png`

Figure 5.20: S10: Map Popup Detail View

Screenshot Placeholder

Missing file: `../../images/screenshots/S11_public_board_desktop.png`

Figure 5.21: S11: Public Grievance Board

Screenshot Placeholder

Missing file: `../../images/screenshots/S12_heatmap_desktop.png`

Figure 5.22: S12: Grievance Heatmap Page

Screenshot Placeholder

Missing file: `../../images/screenshots/S13_profile_admin_desktop.png`

Figure 5.23: S13: Profile Page (Admin)

Screenshot Placeholder

Missing file: `../../images/screenshots/S14_citizen_dashboard_desktop.png`

Figure 5.24: S14: Citizen Dashboard with Timeline

Screenshot Placeholder

Missing file: `../../images/screenshots/S15_notifications_panel_desktop.png`

Figure 5.25: S15: Notifications Dropdown Panel

5.5.2 Supplementary Screens (Optional in Final Print)

Screenshot Placeholder

Missing file:
`../../images/screenshots/SX1_file_complaint_upload_desktop.png`

Figure 5.26: SX1: File Complaint Upload Interface

Screenshot Placeholder

Missing file:
`../../images/screenshots/SX2_citizen_timeline_multilingual_desktop.png`

Figure 5.27: SX2: Multilingual Citizen Timeline Snapshot

Screenshot Placeholder

Missing file:
`../../images/screenshots/SX3_resolved_feedback_snapshot_desktop.png`

Figure 5.28: SX3: Resolved Complaint with Citizen Feedback

Screenshot Placeholder

Missing file:
`../../images/screenshots/SX4_result_location_analysis_desktop.png`

Figure 5.29: SX4: Result Page with Location Context and AI Analysis

Screenshot Placeholder

Missing file:
`../../images/screenshots/SX5_result_grievance_description_desktop.png`

Figure 5.30: SX5: Result Page with Grievance Description Editing

5.6 Chapter Summary

This chapter defined structural design choices, module interactions, data modeling strategy, and UI architecture. The design emphasizes traceability, secure operations, and practical maintainability while accommodating AI-assisted workflows in a municipal governance context.

Chapter 6: Implementation Planning

Implementation planning for Jan-Sunwai AI focused on three parallel goals: reproducibility, maintainable module growth, and deployment readiness. The implementation strategy treated backend, frontend, AI runtime, and operational scripts as coordinated components rather than isolated code tracks.

6.1 Implementation Environment

6.1.1 Local Development Environment

The local workflow supports rapid iteration with role-based UI validation and backend testing. Typical development topology includes:

- Backend running on FastAPI/Uvicorn at port 8000.
- Frontend development server on Vite at port 5173.
- MongoDB either local service or containerized dependency.
- Ollama runtime with configured model set on host machine.

The local setup was selected to reduce iteration latency while preserving fidelity with production behavior through shared APIs, environment variables, and health semantics.

6.1.2 Production-Style Container Environment

For deployment simulation and handover readiness, the project includes a unified compose configuration with a production profile. In production mode:

- MongoDB runs as a dedicated service with health checks.
- Backend runs from production Dockerfile and mounts uploads volume to `/app/uploads`.
- Frontend runs in Nginx container under compose profile `prod`.
- Health checks and log rotation policies are explicitly configured.

This ensures that local verification and deployment handover use the same operational assumptions.

6.2 Tools and Technologies Used

Table 6.1: Implementation Toolchain and Use Cases

Category	Tool/Library	Implementation Use
Backend Framework	FastAPI	Route definitions, dependency injection, validation, middleware pipeline
Data Validation	Pydantic	Request/response schema validation and structured models
Auth and Security	python-jose, passlib[bcrypt]	JWT token issuance/validation and password hashing
HTTP Testing	httpx	Async API smoke and integration checks

Category	Tool/Library	Implementation Use
Database Driver	Motor/PyMongo stack	MongoDB async operations and collection access
AI Runtime	Ollama	Local vision and reasoning model invocation
Image Handling	Pillow	Upload payload validation and EXIF processing
Frontend Framework	React	Role-aware SPA component and page architecture
Build Tool	Vite	Fast frontend development and optimized production build
Client HTTP	Axios	Frontend API communication layer
Routing	react-router-dom	Protected routes and role-based navigation
Containerization	Docker + Compose	Reproducible orchestration of backend/frontend/database services
QA Automation	pytest, locust	Unit/integration/security/resilience/load test execution
Documentation	Markdown + LaTeX	Runbooks, architecture docs, and final academic report

6.3 Coding Standards Followed

6.3.1 Backend Standards

- Router-based modularization for domain isolation.
- Strict schema validation through typed request models.
- Clear HTTP semantics for validation, auth, and lifecycle errors.
- Service extraction for cross-cutting concerns (assignment, sanitization, storage, email).
- Structured logging and failure-safe response behavior.

6.3.2 Frontend Standards

- Role-aware route guards and dashboard redirection.
- Separation of pages, reusable components, layouts, and context providers.
- Consistent API base usage through environment variables.
- Explicit loading/error states for user-facing long-running operations.
- Responsive design checks for desktop and mobile interaction baselines.

6.3.3 Configuration Standards

- Single canonical backend environment file for consistency.
- Explicit production defaults and overrides for sensitive settings.
- Avoidance of duplicate compose and env artifacts to reduce drift.
- Documented scripts for startup, tests, deployment simulation, and verification.

6.4 Implementation Work Packages

6.4.1 Work Package 1: Core Backend and Data Layer

Deliverables included app bootstrap, database connection lifecycle, auth primitives, user and complaint schemas, and initial complaint CRUD operations. This package established foundational APIs and validation behavior.

6.4.2 Work Package 2: AI Pipeline Integration

Deliverables included classifier orchestration, model fallback strategy, rule-engine scoring, queue-based generation path, and timing metadata in analysis responses. This package transformed basic upload capability into intelligent intake flow.

6.4.3 Work Package 3: Frontend Role Flows

Deliverables included authentication screens, analyze/result pathway, role dashboards, notifications UI, and profile controls. Route guards and dashboard routing ensured role-appropriate user journeys.

6.4.4 Work Package 4: Operational Features

Deliverables included worker assignment logic, timeline/audit features, comments/notes, escalation, triage, analytics, and CSV exports. This package completed department-level and governance-level operational requirements.

6.4.5 Work Package 5: Security and Reliability Hardening

Deliverables included upload integrity checks, sanitization, security headers, CORS tightening, rate-limiting toggles, readiness/liveness checks, graceful AI degradation, and resilience-focused tests.

6.4.6 Work Package 6: Deployment and Handover Readiness

Deliverables included production compose profile, deployment guides, API references, backup scripts, and verification procedures. This package supported transition from development artifact to deployable product.

6.5 Implementation Sequence and Dependencies

Table 6.2: Implementation Sequence with Dependency Notes

Step	Implementation Activity	Dependency Consideration
1	Establish backend skeleton and DB connectivity	Required before auth and complaint persistence
2	Build auth/session primitives	Required before protected API and role-specific frontend routes
3	Implement analyze pipeline and storage handling	Required before citizen intake flow can be tested end-to-end
4	Build frontend analyze/result workflow	Depends on stable analyze payload and auth behavior
5	Add assignment and lifecycle controls	Depends on complaint schema maturity and worker metadata
6	Add triage/analytics and governance endpoints	Depends on reliable complaint state and metadata richness
7	Add security and reliability hardening	Should follow core flow stability to avoid hidden regressions
8	Run tests and UAT cycles	Depends on complete feature slices and stable environments
9	Prepare production profile and handover docs	Depends on validated security/performance baseline

6.6 Configuration and Secret Planning

The implementation plan emphasized explicit secret handling and runtime configurability. Important control groups include:

- Authentication controls: JWT secret, algorithm, token expiry.
- Data controls: Mongo URL and database selection.
- AI controls: model identifiers, timeout windows, queue worker count.
- Security controls: CORS origin allowlist and rate-limiter mode.
- Communication controls: SMTP host/port/from for notification relay.

A notable planning correction during implementation was moving to one canonical backend environment file to eliminate drift between local and production templates.

6.7 Quality Gates in Implementation Plan

Table 6.3: Implementation Quality Gates

Gate	Acceptance Criterion
Functional API baseline	Health endpoints and core auth routes return expected responses
Analyze pipeline gate	Valid image upload produces structured classification and draft status output
Lifecycle gate	Complaint create/update/assignment/notification chain works end-to-end
Security gate	Upload hardening, sanitization, role checks, and headers validated in tests
Reliability gate	Graceful fallback behavior under model/DB faults verified
Deployment gate	Compose stack health checks pass and service topology is reproducible
UAT gate	Citizen and admin scripts complete without blocking defects
Documentation gate	Deployment, API, user manual, and report artifacts synchronized

6.8 Resource and Effort Planning

Implementation effort was distributed between development, stabilization, testing, and documentation. Rather than pushing all testing to the end, the plan incorporated recurring verification cycles after major feature groups. This reduced late-stage uncertainty and shortened defect turnaround time.

6.9 Detailed Module Implementation Notes

6.9.1 Backend Router Implementation Notes

Table 6.4: Router-Level Implementation Planning Notes

Router Group	Core Responsibilities	Planning/Implementation Considerations
users	registration, login, profile update, password reset	Required generic forgot-password response to prevent account enumeration; reset tokens persisted as hashes with expiry lifecycle
complaints	analyze, create, list, detail, update, transfer, escalate	Needed consistent status-history append behavior and role-aware update guards; analyze route required graceful 503 degrade path

Router Group	Core Responsibilities	Planning/Implementation Considerations
workers	assignment views, approval/rejection, service-area updates	Needed separation between worker self-actions and admin governance actions; assignment diagnostics endpoint added for ops clarity
notifications	list, unread count, mark single/all read	Badge consistency required deterministic unread counter behavior and ownership checks
triage	review queue and decision endpoint	Admin-only corrective path for low-confidence AI cases; decision audit notes required
analytics/public	dashboards, heatmap, anonymized public listing	Query aggregation planning required pagination defaults and safe exposure model for public feed
health	live/ready/models/gpu probes	Needed operationally meaningful readiness semantics and low-overhead liveness behavior

6.9.2 Service-Layer Planning Notes

Table 6.5: Service-Level Implementation Planning

Service	Planning Note
Assignment service	Haversine-based distance filtering was planned with fallback behavior when location metadata is missing
Storage service	Validation pipeline designed as extension + content-type + magic-number + payload decode sequence
Sanitization service	Planned as reusable utility to keep payload cleaning centralized and testable
LLM queue service	Introduced to absorb long-running draft generation and preserve responsive user experience
Escalation loop	Planned as background process with authority parent mapping and timeline updates
Email service	Implemented as environment-driven stub/relay abstraction for gradual production integration

6.10 Deployment Runbook and Rollout Plan

6.10.1 Rollout Stages

Table 6.6: Deployment Rollout Stages

Stage	Objective	Execution Checklist
R1	Environment readiness	Verify host runtime versions, model availability, and environment variables
R2	Service bootstrap	Start MongoDB and backend; validate health/live and health/ready probes
R3	Frontend linkage	Start frontend/proxy path and verify API base routing behavior
R4	Functional smoke	Execute login, analyze, complaint creation, and notification badge checks
R5	Governance smoke	Validate admin triage, worker assignment controls, and export flow
R6	Hardening validation	Execute security/resilience test scripts and capture baseline metrics
R7	Handover sign-off	Archive logs, backups, and runbook evidence for deployment package

6.10.2 Rollback Planning

Rollback planning was included to avoid high-impact downtime during deployment changes:

- Preserve previous container images and compose configuration snapshots.
- Ensure database backup is taken before schema/index-affecting changes.
- Revert frontend image and backend service independently when only one tier regresses.
- Maintain known-good environment variable snapshot for rapid restoration.

6.11 Operational Monitoring Plan

Table 6.7: Operational Monitoring Matrix

Signal	Collection Source	Operational Interpretation
API process time	Response header `X-Process-Time`	Detect slow endpoints and regressions after deployments
Readiness status	/health/ready endpoint	Indicates dependency-level degradation, mainly database health
Model availability	/health/models endpoint	Confirms runtime model inventory and inference readiness
GPU/model state	/health/gpu endpoint	Indicates active model and VRAM-related load behavior

Signal	Collection Source	Operational Interpretation
Notification backlog	unread-count behavior and list volume	Detects delayed event propagation or read-state anomalies
Assignment queue growth	open/unassigned complaints and worker status	Helps identify staffing bottlenecks and area mismatch
Triage queue size	low-confidence complaint count	Indicates classifier uncertainty trend and review load
Error logs	rotating backend logs	Captures fault signatures for rapid remediation

6.12 Change Management and Release Strategy

6.12.1 Release Checklist

1. Confirm automated tests pass for backend and frontend quality gates.
2. Verify compose validation for default and production profiles.
3. Validate security controls: headers, CORS allowlist, upload checks, and role restrictions.
4. Confirm migration/index scripts produce expected database state.
5. Review documentation deltas (API, deployment, report references) before tagging release.
6. Capture release notes with risk statements and rollback pointers.

6.12.2 Implementation Learnings

Implementation planning revealed that operational simplification tasks (for example, consolidating environment and compose artifacts) can have impact comparable to feature work. These tasks reduced recurring integration errors, made CI behavior more predictable, and improved deployment communication across stakeholders.

6.13 Chapter Summary

Implementation planning translated conceptual design into execution-ready work packages, sequence constraints, and quality gates. The plan successfully balanced rapid feature delivery with operational robustness, resulting in a deployable and documentable civic platform.

Chapter 7: Testing

Testing for Jan-Sunwai AI was designed as a layered strategy combining automated checks, manual UAT scripts, resilience drills, and security/load validation. Because the platform supports governance-sensitive workflows, testing emphasized not only feature correctness but also operational integrity under degraded conditions.

7.1 Testing Plan

7.1.1 Testing Objectives

- Validate correctness of core APIs and role authorization boundaries.
- Verify AI-path behavior for valid, invalid, and failure scenarios.
- Confirm notification chain consistency across complaint lifecycle changes.
- Validate resilience behavior under model/database disruptions.
- Ensure production-readiness through integration, load, and UAT checks.

7.1.2 Test Environment

Table 7.1: Testing Environment Baseline

Environment Aspect	Configuration
Backend runtime	FastAPI application with test clients and dependency overrides
Database mode	MongoDB-backed flows with selective mock/fake collection tests
Frontend mode	Vite build/lint validation and persona-driven manual walkthroughs
Model runtime mode	Ollama available for normal flows; manually disabled for resilience checks
Automation tools	pytest, httpx, locust, script wrappers for platform-specific execution

7.1.3 Coverage Matrix

Table 7.2: Testing Coverage Matrix by Suite

Test Artifact	Type	Coverage Highlights
`test_schemas.py`	Unit Validation	GeoLocation bounds, complaint schema validity, confidence-score guardrails
`test_auth_permissions.py`	Unit Authorization	Token claims, admin and worker/dept-head access guards

Test Artifact	Type	Coverage Highlights
`test_api_integration.py`	Integration Smoke	Root route, health checks, versioned aliases, lightweight CI path
`test_api_matrix.py`	Integration Matrix	/api/v1 profile route, CORS preflight behavior, security header presence
`test_notification_chain.py`	Integration Behavior	Status update notification creation, email hook path, mark-all-read logic
`test_resilience_security.py`	Resilience + Security	Upload hardening, analyze degrade 503, readiness degradation, sanitization
Manual UAT scripts	User Acceptance	Citizen and admin journeys, friction logging, UX verification
Load scripts (locust)	Performance	Endpoint mix throughput and graceful degradation behavior

7.2 Types of Testing

7.2.1 Unit Testing

Unit tests focused on isolated validation and control logic.

Table 7.3: Representative Unit Test Cases

Case ID	Area	Expected Behavior
UT-01	GeoLocation bounds	Reject latitude outside $[-90, 90]$ and longitude outside $[-180, 180]$
UT-02	AI confidence schema	Reject confidence values outside $[0, 1]$
UT-03	Admin role check	Non-admin caller receives HTTP 403 on admin-only dependency
UT-04	Worker approval gate	Unapproved worker cannot pass worker-only dependency path
UT-05	Sanitization function	Script payload transformed to escaped safe representation

7.2.2 Integration Testing

Integration tests verified route composition, middleware behavior, and cross-router guarantees such as versioned aliases and headers.

- Root and liveness endpoints confirmed service online state.
- Versioned alias endpoints under `/api/v1` validated integration compatibility.
- Security headers confirmed on endpoint responses.

- Preflight CORS behavior validated for allowed origins.

7.2.3 System Testing

System testing executed end-to-end flows across authentication, analyze, complaint lifecycle, assignment, and notification propagation. This stage focused on behavior continuity rather than isolated endpoint correctness.

Table 7.4: System Test Scenarios

Scenario	Workflow	Pass Condition
ST-01	Citizen upload -> analyze -> submit	Complaint record created with status history initialized
ST-02	Dept-head status update	Citizen receives notification and timeline reflects transition
ST-03	Worker marks complaint done	Complaint transitions and worker slot logic updates as expected
ST-04	Admin triage decision	Ambiguous complaint receives governed label decision path
ST-05	CSV export	Export file generated with expected schema columns

7.2.4 User Acceptance Testing (UAT)

The UAT plan was executed through persona scripts:

- Citizen script: login, analyze, map review, submit, dashboard verification.
- Admin script: queue filtering, bulk actions, heatmap view, reassignment, CSV export.
- Feedback loop: friction capture with severity ranking and follow-up fixes.
- Resilience check: AI service interruption and recovery behavior in UI.

7.3 Testing Techniques

7.3.1 Black-Box Behavioral Testing

Primary API workflows were tested against expected request/response contracts without depending on internal implementation details. This improved confidence in external behavior from a consumer perspective.

7.3.2 Negative Testing

Negative paths were intentionally exercised:

- Invalid tokens and unauthorized role access.
- Corrupted/mismatched file uploads.
- Expired/invalid reset token paths.
- Model/runtime failure leading to controlled error responses.

7.3.3 Resilience and Fault Injection

Resilience tests simulated:

- Ollama unavailability during analyze requests.
- Database ping failures for readiness route validation.
- Corrupted payload uploads to confirm robust rejection semantics.

7.3.4 Performance and Load Testing

Load-testing scripts executed mixed endpoint traffic where read-heavy routes and notification checks dominate, with controlled analyze pressure to assess graceful degradation behavior.

7.4 Defect Logging

7.4.1 Defect Classification

Defects were classified by severity and functional impact:

- High: Breaks core flow or security boundary.
- Medium: Degrades key workflow but has workaround.
- Low: Cosmetic or minor UX issue.

7.4.2 Defect Log Snapshot

Table 7.5: Defect Log Snapshot and Resolution

Defect ID	Severity	Observed Issue	Resolution Summary
D-001	High	Login dependency crash in CI due to incompatible form dependency path	Reworked login endpoint to explicit form fields and validated with full backend tests
D-002	Medium	Workflow runtime warnings from deprecated Node 20 action dependencies	Updated GitHub workflow actions to newer major versions compatible with current runtime
D-003	Medium	Upload volume misconfiguration created nested backend/backend/uploads path	Corrected compose volume target to /app/uploads and cleaned stray nested folder artifacts
D-004	Medium	Missing frontend forgot-password discoverability path	Added dedicated forgot/reset pages, route wiring, and contextual links in login/navbar/home

Defect ID	Severity	Observed Issue	Resolution Summary
D-005	Low	Placement of forgot-password link reduced form clarity	Repositioned link under password field for better UX coherence

7.5 Testing Outcomes

Table 7.6: Outcome Summary by Validation Area

Validation Area	Outcome	Notes
Backend automated suite	Pass	Core suite executed successfully with latest observed run showing full pass status
Security checks	Pass	Upload hardening, sanitization, and authorization boundaries validated
Resilience checks	Pass	Controlled degraded responses confirmed for model/DB disruption paths
Frontend lint/build	Pass	UI changes and route additions validated through build and lint cycle
Compose validation	Pass	Unified compose configuration validated for default and prod profile flows
UAT persona scripts	Pass with refinements	Minor UX adjustments applied in same sprint feedback loop

7.6 Residual Risks and Test Gaps

Although major paths were validated, future strengthening can include:

- Long-duration soak tests on production-equivalent infrastructure.
- Expanded browser matrix for UI behavior consistency.
- External vulnerability scanning integration in release pipeline.
- Higher-volume synthetic image corpus evaluation for classifier drift tracking.

7.7 Detailed UAT Case Matrix

Table 7.7: Persona-Driven UAT Matrix

Case ID	Persona	Scenario	Priority	Expected Result
UAT-01	Citizen	Register new citizen account and complete first login	High	Account created and token-based session starts
UAT-02	Citizen	Submit valid image through analyze flow	High	Classification and draft payload are returned

Case ID	Persona	Scenario	Priority	Expected Result
UAT-03	Citizen	Edit draft text and submit complaint	High	Complaint record appears with initial Open status
UAT-04	Citizen	View dashboard timeline after submission	High	Status history includes creation event
UAT-05	Citizen	Open notifications panel and unread counter	Medium	Unread count and list are synchronized
UAT-06	Citizen	Forgot-password and reset-password flow	High	Password reset succeeds with valid token
UAT-07	Worker	Login as approved worker and view assigned tasks	High	Only assigned tasks and worker actions are visible
UAT-08	Worker	Mark assigned complaint as done	High	Complaint transitions and worker slot is updated
UAT-09	Dept Head	View department queue and filter by status	High	Department-scoped records only
UAT-10	Dept Head	Update complaint status with note	High	Status history and citizen notification are generated
UAT-11	Dept Head	Attempt update on foreign department complaint	High	Access denied with authorization error
UAT-12	Admin	Approve pending worker registration	High	Worker status changes to approved
UAT-13	Admin	Perform manual worker assignment	High	Assignment reflected in complaint and worker workload
UAT-14	Admin	Open triage queue and approve/reject item	High	Decision persisted and queue updates accordingly

Case ID	Persona	Scenario	Priority	Expected Result
UAT-15	Admin	Execute bulk status update	Medium	All selected complaints updated consistently
UAT-16	Admin	Export complaints as CSV	Medium	Download file with expected column structure
UAT-17	Admin	View analytics and heatmap screens	Medium	Aggregates and map points render without breakage
UAT-18	Public user	Access public board without authentication	Medium	Anonymized listing visible without sensitive fields
UAT-19	Citizen	Submit malformed image payload	High	Request rejected with safe validation message
UAT-20	Citizen	Trigger analyze while model unavailable	High	User-friendly retryable 503 response path

7.8 Load Benchmark Profiles and Acceptance Rubric

Table 7.8: Load Profile and Acceptance Guidance

Profile	Users	Spawn Rate	Duration	Acceptance Guidance
Smoke	20	5/s	5 min	No critical endpoint failure; stable liveness and predictable response envelopes
Baseline	70	10/s	15 min	Controlled latency growth; no persistent failure mode in public and health endpoints
Peak	120	20/s	20 min	Graceful degradation accepted for analyze path (429/503) while core status pages remain functional

7.9 Security Regression Matrix

Table 7.9: Security Regression Verification Matrix

Sec ID	Control Under Test	Expected Code	Validation Criterion
SEC-01	Invalid bearer token on protected complaint route	401	Unauthorized access blocked
SEC-02	Non-admin call on admin endpoint	403	Role boundary enforced
SEC-03	Malformed JPEG payload with .jpg extension	400	Header mismatch rejected
SEC-04	Oversized upload payload	413	Max file-size rule enforced
SEC-05	XSS payload in note/comment text	200 with escaped output	Sanitization removes executable markup
SEC-06	Notification read on foreign ownership ID	404	Ownership scope preserved
SEC-07	Missing security headers check	200 with headers present	nosniff, frame deny, CSP and policy headers included
SEC-08	CORS preflight from allowed origin	200/204	Allow-origin policy consistent with configuration
SEC-09	Invalid/expired reset token usage	400	Reset workflow rejects unsafe token state
SEC-10	Analyze under runtime failure	503	Controlled degrade message returned without server crash

7.10 Chapter Summary

Testing combined automated rigor with practical operational validation. The resulting quality posture supports both technical confidence and governance readiness, while documented residual risks provide a clear roadmap for post-submission hardening.

Chapter 8: Limitations and Future Scope

No production-oriented civic platform is complete without explicit recognition of current boundaries. Jan-Sunwai AI delivers a strong operational baseline, but several limitations remain due to time, infrastructure, and deployment-context constraints.

8.1 Current Limitations

8.1.1 Model and AI Limitations

- Classification quality depends on local model behavior and image quality.
- Ambiguous civic scenes may still require administrative triage despite hybrid logic.
- Domain adaptation is limited to existing taxonomy and keyword/rule coverage.
- Draft generation quality can vary by language complexity and scene context.

8.1.2 Infrastructure Limitations

- Throughput is constrained by local hardware, especially VRAM and CPU availability.
- Single-node deployment is suitable for pilot and controlled workloads, not large multi-zone production scale without additional orchestration.
- Current queue strategy is in-process and may require distributed message infrastructure for higher concurrency targets.

8.1.3 Workflow and Data Limitations

- Worker service-area quality depends on correctness and completeness of admin-maintained profiles.
- Escalation semantics are configurable but not yet integrated with external authority APIs.
- Public feed is intentionally anonymized, limiting deeper citizen-facing analytics.
- Historical trend analytics can be expanded with richer time-series and SLA breach segmentation.

8.1.4 Security and Governance Limitations

- Security checks are strong at application layer but external perimeter controls (WAF, zero-trust segmentation) are deployment-dependent.
- Email delivery remains stub-based until institutional SMTP integration is finalized.
- Continuous dependency vulnerability tracking is not yet fully automated in release workflow.

8.2 Future Scope

8.2.1 AI and Intelligence Enhancements

1. Introduce domain fine-tuning or retrieval-augmented context for better local scene understanding.

2. Add multilingual voice-to-text intake for accessibility.
3. Integrate confidence calibration dashboards and model drift alerts.
4. Build image deduplication and duplicate complaint clustering using visual embeddings.

8.2.2 Operational Workflow Enhancements

1. Implement SLA policy engine with configurable thresholds per department and issue class.
2. Add escalation automation with conditional approvals and supervisor override logs.
3. Extend assignment with route optimization and field workload balancing by travel cost.
4. Introduce richer citizen feedback loops and resolution quality scoring.

8.2.3 Platform and Deployment Enhancements

1. Move from single-node compose to orchestrated deployments for higher availability.
2. Add asynchronous message broker for queue resilience and horizontal worker scaling.
3. Integrate centralized observability stack for metrics, traces, and security audit events.
4. Implement backup verification drills and disaster recovery runbooks with recovery point objectives.

8.2.4 Security and Compliance Enhancements

1. Introduce periodic penetration testing workflow with report tracking.
2. Add secret rotation automation and hardware-backed key management integration.
3. Expand privacy controls for retention and redaction policies aligned with institutional mandates.
4. Include compliance templates for municipal procurement and audit reporting.

8.3 Roadmap Perspective

Table 8.1: Indicative Post-Submission Roadmap

Horizon	Priority Area	Expected Deliverables
0–3 months	Stabilization and hardening	SMTP integration, expanded tests, deployment checklist closure
3–6 months	AI and workflow maturity	Better triage intelligence, SLA policy engine, richer analytics
6–12 months	Scale and institutionalization	Distributed queueing, observability stack, multi-zone deployment model
12+ months	Advanced governance intelligence	Predictive demand analytics and department workload forecasting

8.4 Risk-Prioritized Improvement Backlog

Table 8.2: Future Backlog Prioritization Matrix

ID	Improvement Item	Impact	Effort	Suggested Horizon
FB-01	SMTP relay production integration with delivery audit logs	High	Medium	0–3 months
FB-02	Centralized secrets management and rotation policy	High	Medium	0–6 months
FB-03	Distributed queue broker for LLM jobs	High	High	3–9 months
FB-04	Expanded language support in generated drafts	Medium	Medium	3–6 months
FB-05	Domain-adapted classifier tuning dataset	High	High	6–12 months
FB-06	Duplicate complaint detection using embeddings	Medium	High	6–12 months
FB-07	SLA rules engine with department-specific thresholds	High	Medium	3–9 months
FB-08	Escalation policy simulator for governance review	Medium	Medium	6–12 months
FB-09	Citizen voice intake and transcription	Medium	High	9–15 months
FB-10	Geo-aware workload balancing with travel-time estimation	Medium	High	6–12 months
FB-11	Real-time websocket notification stream	Medium	Medium	3–6 months
FB-12	Expanded public analytics visualizations	Medium	Medium	3–9 months
FB-13	Multi-tenant authority hierarchy support	High	High	9–18 months
FB-14	Formal accessibility conformance audit (WCAG)	High	Medium	0–6 months
FB-15	Integrated vulnerability scanning in CI pipeline	High	Low	0–3 months
FB-16	Observability stack with trace correlation	High	Medium	3–9 months
FB-17	Offline-first synchronization for field staff app mode	Medium	High	9–18 months
FB-18	Complaint closure quality scoring and follow-up loop	Medium	Medium	3–9 months

ID	Improvement Item	Impact	Effort	Suggested Horizon
FB-19	Auto-generated administrative weekly digest reports	Low	Low	0–3 months
FB-20	Predictive hotspot forecasting for civic incidents	Medium	High	12+ months

8.5 Adoption Considerations for Municipal Deployment

8.5.1 Institutional Readiness Factors

Municipal deployment success is influenced by organizational readiness as much as software maturity. Key factors include role training quality, escalation policy clarity, and data-governance ownership. Even technically complete systems can underperform if operational owners do not have clear process responsibility.

8.5.2 Change Management Considerations

- Staff onboarding must include role-specific workflow rehearsals, not only UI tours.
- Escalation policy language should be standardized before production launch.
- Existing manual grievance channels should run in parallel during pilot transition.
- Citizen communication templates should be localized and reviewed for clarity.

8.5.3 Data Governance Considerations

Future rollout should include an explicit policy for retention, redaction, access audits, and report publication limits. This is particularly important where complaint evidence may include personal or location-sensitive context.

8.6 Research Continuation Opportunities

The platform also creates opportunities for academic extension:

1. Comparative analysis of hybrid rule+LLM pipelines versus end-to-end multimodal classifiers in civic datasets.
2. Measurement of complaint resolution outcomes before and after AI-assisted intake.
3. Bias and fairness analysis across department categories and geographies.
4. Human-in-the-loop triage ergonomics and decision consistency studies.

8.7 Chapter Summary

The current system is robust for pilot and structured deployment but remains intentionally conservative in distributed scaling and advanced automation. The future scope presented here provides a realistic pathway from project-grade implementation to institutional-grade civic infrastructure.

Chapter 9: Conclusion and References

9.1 Conclusion

Jan-Sunwai AI demonstrates that a practical civic grievance platform can combine AI assistance with deterministic governance controls without sacrificing transparency or operational safety. The technology choices and implementation baseline align with established platform documentation for FastAPI, React, MongoDB, and Docker [1, 2, 3, 8]. The project delivered:

- A complete complaint lifecycle from image-based intake to resolution tracking.
- Role-specific workflows for citizens, workers, department heads, and administrators.
- A hybrid classification pipeline that improves routing stability through staged decision logic.
- Security and reliability controls aligned with deployment realities.
- Containerized deployment and documentation artifacts suitable for institutional handover.

From an engineering standpoint, the most significant contribution is the architecture pattern: use AI where it adds value, but preserve deterministic checkpoints and human governance where accountability is essential. This pattern is especially appropriate in public-service software, where explainability and auditability are as important as automation speed. Security controls for upload validation, input handling, and operational hardening were shaped by OWASP guidance [6, 7].

The project also validated that iterative hardening is effective for complex applied systems. Functional breadth, security controls, resilience behavior, and deployment readiness were improved incrementally, with testing and documentation integrated into delivery rather than deferred to the end. Performance and scenario-load behavior were evaluated using the Locust framework [9].

As a final outcome, Jan-Sunwai AI is positioned as a strong foundation for municipal deployment pilots. With further scaling and policy integration, it can evolve into a wider civic operations platform supporting higher throughput, stronger compliance workflows, and richer planning intelligence.

References

[1] FastAPI, "FastAPI documentation," [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Apr. 7, 2026].

[2] React, "React documentation," [Online]. Available: https://react.dev. [Accessed: Apr. 7, 2026].

[3] MongoDB, "MongoDB documentation," [Online]. Available: https://www.mongodb.com/docs. [Accessed: Apr. 7, 2026].

[4] Pydantic, "Pydantic documentation," [Online]. Available: https://docs.pydantic.dev. [Accessed: Apr. 7, 2026].

[5] Ollama, "Ollama documentation," [Online]. Available: https://ollama.com. [Accessed: Apr. 7, 2026].

[6] OWASP Foundation, "OWASP Top 10: Web application security risks," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 7, 2026].

[7] OWASP Foundation, "File upload security cheat sheet," [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html. [Accessed: Apr. 7, 2026].

[8] Docker, "Docker documentation," [Online]. Available: https://docs.docker.com. [Accessed: Apr. 7, 2026].

[9] Locust, "Locust documentation," [Online]. Available: https://docs.locust.io. [Accessed: Apr. 7, 2026].

[10] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[11] Jan-Sunwai AI Project Team, "Jan-Sunwai AI project documentation set (API reference, architecture notes, UAT plan, security testing guide, and timeline artifacts)," repository documentation, 2026.

Chapter 10: Appendices

10.1 Appendix A: API Endpoint Snapshot

This appendix summarizes the primary endpoint inventory implemented in the current release. All major endpoint groups are additionally exposed under the `/api/v1` alias path.

Table 10.1: API Endpoint Snapshot

Method	Endpoint	Auth	Purpose
POST	`/users/register`	No	Citizen/worker self-registration
POST	`/users/login`	No	JWT token issuance
GET	`/users/me`	Yes	Current user profile
PATCH	`/users/me`	Yes	Profile update
POST	`/users/forgot-password`	No	Reset token issuance request
POST	`/users/reset-password`	No	Password reset submission
POST	`/analyze`	Yes	Image analysis and draft initiation
POST	`/analyze/regenerate`	Yes	Regenerate complaint draft
GET	`/complaints/generation/{job_id}`	Yes	Poll queued generation status
POST	`/complaints`	Yes	Create complaint
GET	`/complaints`	Yes	Role-filtered complaint list
GET	`/complaints/{complaint_id}`	Yes	Complaint detail
PATCH	`/complaints/{complaint_id}/status`	Admin/Dept Head	Status transition
PATCH	`/complaints/{complaint_id}/transfer`	Admin/Dept Head	Department transfer
POST	`/complaints/{complaint_id}/escalate`	Admin/Dept Head	Authority escalation
POST	`/complaints/{complaint_id}/feedback`	Citizen owner	Resolution feedback
POST	`/complaints/{complaint_id}/notes`	Admin/Dept Head	Internal department notes
GET	`/complaints/{complaint_id}/notes`	Admin/Dept Head	Fetch department notes
POST	`/complaints/{complaint_id}/comments`	Yes	Add collaborative comment

Method	Endpoint	Auth	Purpose
GET	/complaints/{complaint_id}/comments	Yes	Read collaborative thread
POST	/complaints/bulk/status	Admin	Bulk status updates
POST	/complaints/bulk/transfer	Admin	Bulk transfer updates
GET	/complaints/export/csv	Admin	CSV export
GET	/workers/me	Worker	Worker profile and workload
PATCH	/workers/me/status	Worker	Availability update
PATCH	/workers/me/complaints/{complaint_id}/done	Worker	Mark assigned complaint done
GET	/workers	Admin	List workers (pending/all)
GET	/workers/my-department	Admin/Dept Head	Department worker listing
PATCH	/workers/{worker_id}/approve	Admin	Approve worker
DELETE	/workers/{worker_id}/reject	Admin	Reject pending worker
POST	/workers/{worker_id}/assign/{complaint_id}	Admin	Manual assignment
PATCH	/workers/{worker_id}/area	Admin	Update worker service area
GET	/workers/assignment-debug	Admin	Assignment diagnostics
POST	/workers/reassign-unassigned	Admin	Retry assignment for open queue
GET	/notifications	Yes	Notification list
GET	/notifications/unread-count	Yes	Unread badge count
PATCH	/notifications/{notification_id}/read	Yes	Mark single notification read
PATCH	/notifications/read-all	Yes	Mark all notifications read
GET	/triage/review-queue	Admin	Retrieve low-confidence queue
POST	/triage/review-queue/decision	Admin	Apply triage decision
GET	/analytics/overview	Admin	Overview metrics
GET	/analytics/heatmap	Admin/Dept Head	Geospatial complaint distribution
GET	/public/complaints	No	Public anonymized complaint feed
GET	/health/live	No	Liveness probe
GET	/health/ready	No	Readiness probe
GET	/health/models	No	Model availability status
GET	/health/gpu	No	GPU/model load status

10.2 Appendix B: Environment Variable Catalog

Table 10.2: Core Backend Environment Variables

Variable	Typical Default	Purpose
APP_ENV	development	Runtime mode (development/production)
MONGODB_URL	mongodb://localhost:27017	Primary Mongo connection URI
MONGO_URL	mongodb://localhost:27017	Legacy fallback connection URI
DB_NAME	jan_sunwai_db	Database name
JWT_SECRET_KEY	change-me-in-production	JWT signing secret
JWT_ALGORITHM	HS256	JWT signing algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	1440 (dev) / 480 (prod)	Token expiration minutes
RATE_LIMIT_ENABLED	false (dev)	Rate-limiting toggle
ALLOWED_ORIGINS	localhost origins list	CORS allowlist
OLLAMA_BASE_URL	http://localhost:11434	Ollama API endpoint
VISION_MODEL	qwen2.5vl:3b	Primary vision model
MID_VISION_MODEL	granite3.2-vision:2b	Mid-tier vision fallback
FALLBACK_VISION_MODEL	granite3.2-vision:2b	Final vision fallback
REASONING_MODEL	llama3.2:1b	Reasoning/drafting model
VISION_TIMEOUT_SECONDS	240	Vision tier timeout
LLM_INLINE_TIMEOUT_SECONDS	15	Inline generation wait time
LLM_QUEUE_WORKERS	2	Background generation workers
RULE_ENGINE_ONLY	false	Skip reasoning model when true
AMBIGUITY_THRESHOLD	2.0	Rule-engine ambiguity boundary
UNLOAD_AFTER_REASONING	true	Attempt model unload after use
MODEL_UNLOAD_TIMEOUT_SECONDS	30	Model unload timeout
MODEL_UNLOAD_POLL_INTERVAL_SECONDS	0.1	Poll interval for unload check
KEEP_REASONING_MODEL_WARM	false	Keep reasoning model loaded toggle
COMPLAINT_OUTPUT_MODE	email	Draft output style mode
SMTP_HOST	empty	SMTP relay host
SMTP_PORT	587	SMTP relay port
SMTP_FROM	noreply@jan-sunwai.local	Sender identity for email stub/relay
DEFAULT_PAGE_SIZE	25	Default pagination size
MAX_PAGE_SIZE	100	Maximum pagination limit

10.3 Appendix C: Canonical Department Taxonomy and Routing

Table 10.3: Department to Authority Mapping Snapshot

Department Label	Authority ID	Escalation Parent
Health Department	MUNI_SANITATION	STATE_CMO
Civil Department	MUNI_PWD	STATE_CMO
Horticulture	MUNI_HORTICULTURE	STATE_CMO
Electrical Department	UTIL_DISCOM	STATE_CMO
IT Department	STATE_CMO	CENTRAL_DARPG
Commercial	STATE_CMO	CENTRAL_DARPG
Enforcement	POLICE_TRAFFIC	STATE_CMO
VBD Department	MUNI_SANITATION	STATE_CMO
EBR Department	MUNI_PWD	STATE_CMO
Fire Department	POLICE_LOCAL	STATE_CMO
Uncategorized	Rule/keyword fallback	Nullable

10.4 Appendix D: Script Inventory

Table 10.4: Operational Script Inventory

Script	Purpose
scripts/run_backend.(bat|sh)	Start backend service in local mode
scripts/run_frontend.(bat|sh)	Start frontend service in local mode
scripts/run_tests.(bat|sh)	Execute backend test suite
scripts/run_notification_chain_test.(bat|sh|py)	Validate notification end-to-end behavior
scripts/run_security_test.(bat|sh)	Trigger security-oriented automated checks
scripts/run_resilience_test.(bat|sh)	Execute resilience and fault behavior checks
scripts/run_load_test.(bat|sh|ps1)	Run Locust load profile with target host
scripts/backup_db.(ps1|sh)	MongoDB backup workflow
scripts/verify_prod_stack.(ps1|sh)	Production stack verification checks
scripts/simulate_clean_deploy.(ps1|sh)	End-to-end clean-machine deployment simulation

10.5 Appendix E: Screenshot Asset Checklist

Table 10.5: Screenshot Capture Checklist

ID	Target Screen	Suggested Path
S01	Home hero and CTA	`docs/images/screenshots/S01_home_desktop.png`
S02	Login page	`docs/images/screenshots/S02_login_desktop.png`
S03	Register page	`docs/images/screenshots/S03_register_desktop.png`
S04	Reset password page	`docs/images/screenshots/S04_reset_password_desktop.png`
S05	Admin dashboard overview	`docs/images/screenshots/S05_admin_dashboard_desktop.png`
S06	Triage human review queue	`docs/images/screenshots/S06_triage_review_desktop.png`
S07	Worker assignment panel	`docs/images/screenshots/S07_worker_assignment_panel_desktop.png`
S08	Analytics dashboard	`docs/images/screenshots/S08_analytics_dashboard_desktop.png`
S09	Map overview (satellite)	`docs/images/screenshots/S09_map_overview_satellite_desktop.png`
S10	Map popup detail	`docs/images/screenshots/S10_map_popup_detail_desktop.png`
S11	Public transparency board	`docs/images/screenshots/S11_public_board_desktop.png`
S12	Heatmap page	`docs/images/screenshots/S12_heatmap_desktop.png`
S13	Profile page (admin)	`docs/images/screenshots/S13_profile_admin_desktop.png`
S14	Citizen dashboard with timeline	`docs/images/screenshots/S14_citizen_dashboard_desktop.png`
S15	Notifications dropdown panel	`docs/images/screenshots/S15_notifications_panel_desktop.png`
SX1	File complaint upload screen	`docs/images/screenshots/SX1_file_complaint_upload_desktop.png`
SX2	Multilingual timeline snapshot	`docs/images/screenshots/SX2_citizen_timeline_multilingual_desktop.png`
SX3	Resolved complaint with feedback	`docs/images/screenshots/SX3_resolved_feedback_snapshot_desktop.png`
SX4	Result page location and AI analysis panel	`docs/images/screenshots/SX4_result_location_analysis_desktop.png`
SX5	Result page grievance description editor	`docs/images/screenshots/SX5_result_grievance_description_desktop.png`

10.6 Appendix F: Sample Request and Response Payloads

10.6.1 Login Request (Form Encoded)

```
POST /users/login
Content-Type: application/x-www-form-urlencoded

username=citizen_demo&password=citizen123
```

10.6.2 Analyze Request (Multipart)

```
POST /analyze
Authorization: Bearer <jwt>
Content-Type: multipart/form-data

file=<image file>
language=en
```

10.6.3 Analyze Response (Representative)


```
{
 "classification": {
  "department": "Electrical Department",
  "label": "Electrical Department",
  "confidence": 0.88,
  "method": "hybrid",
  "model_used": "qwen2.5vl:3b"
 },
 "location": {
  "lat": 28.6139,
  "lon": 77.2090,
  "address": "Connaught Place, New Delhi",
  "source": "manual"
 },
 "generated_complaint": "Dear Electrical Department, ...",
 "generation_status": "completed",
 "image_url": "uploads/3a2b....jpg",
 "timings": {
  "vision_ms": 2850.22,
  "rule_engine_ms": 2.11,
  "reasoning_ms": 0.0,
  "total_analyze_ms": 3310.47
 }
}
```


10.6.4 Complaint Creation Payload (Representative)


```
{
```

```
 "description": "Streetlight not working at night.",
 "department": "Electrical Department",
 "image_url": "uploads/3a2b....jpg",
 "location": {
  "lat": 28.6139,
  "lon": 77.2090,
  "address": "Connaught Place, New Delhi",
  "source": "manual"
 },
 "ai_metadata": {
  "confidence_score": 0.88,
  "detected_department": "Electrical Department",
  "labels": ["street light", "dark road"]
 }
}
```


10.6.5 Triage Decision Payload


```
{
 "image": "<complaint_id>",
 "decision": "approved",
 "corrected_label": "Civil Department",
 "note": "Manual override after review"
}
```


10.7 Appendix G: Consolidated Execution Timeline

Table 10.6: Execution Timeline Digest for Final Report Verification

Week	Date Window	Key Completion Events
W1	Jan 28–Jan 31	Repository setup, architecture framing, initial backend bootstrap
W2	Feb 02–Feb 07	Database connectivity, schemas, CRUD baseline, secure upload scaffolding
W3	Feb 09–Feb 14	JWT auth integration, classifier experimentation, dependency stabilization
W4	Feb 16–Feb 21	Geotagging and reverse-geocoding integration with analyze flow
W5	Feb 23–Feb 28	Local model runtime setup and draft generator integration
W6	Mar 02–Mar 07	Frontend role routing and analyze-result integration
W7	Mar 09–Mar 14	Map UX refinement, local draft persistence, end-to-end role walkthrough

Week	Date Window	Key Completion Events
W8	Mar 16–Mar 21	Security hardening: upload checks, sanitization, CORS, headers, indexing
W9	Mar 23–Mar 28	Assignment service, timeline UI, analytics/heatmap, SLA and bulk features
W10	Mar 30–Apr 04	Notification chain, email stub, forgot/reset password, profile editing
W11	Apr 06–Apr 11	Performance optimization, graceful degradation, profiling, resilience checks
W12	Apr 13–Apr 18	Production compose hardening and deployment docs
W13	Apr 20–Apr 25	Expanded tests: unit, integration, security, load, responsiveness
W14	Apr 27–Apr 30	API reference closure, repository cleanup, release candidate preparation
W15	May 04–May 09	Production verification and clean deploy simulation
W16	May 11–May 16	UAT execution and feedback-driven refinements
W17	May 18–May 23	Report and presentation artifact preparation
W18	May 25–May 27	Final submission pack and release closure

10.8 Appendix H: Requirement-to-Test Traceability Matrix

Table 10.7: Traceability from Functional Requirement to Validation Evidence

Req ID	Requirement Theme	Validation Evidence	Status
FR-01	User registration flow	Register route tests and manual registration UI checks	Verified
FR-02	Login and JWT issuance	Auth permission tests and integration smoke login path	Verified
FR-03	Profile editing	API endpoint behavior and profile page walkthrough	Verified
FR-04	Password recovery/reset	Forgot/reset endpoint implementation and UI path checks	Verified
FR-05	Upload validation	Resilience/security tests for header mismatch and size limits	Verified

Req ID	Requirement Theme	Validation Evidence	Status
FR-06	AI-assisted analysis	Analyze endpoint response contract checks and manual scenario runs	Verified
FR-07	Draft regeneration queue	Generation polling route and regeneration request flow	Verified
FR-08	Complaint creation	End-to-end citizen flow and complaint list visibility checks	Verified
FR-09	Role-filtered listing	Role-based list query behavior and dashboard checks	Verified
FR-10	Complaint detail	Complaint detail retrieval and timeline rendering checks	Verified
FR-11	Status update workflow	Notification chain tests and status history assertions	Verified
FR-12	Department transfer	Admin/dept-head transfer operation checks	Verified
FR-13	Escalation support	Escalation endpoint and timeline update behavior checks	Verified
FR-14	Worker assignment	Assignment service behavior and admin assignment controls	Verified
FR-15	Worker task completion	Worker done workflow and slot-free behavior checks	Verified
FR-16	Notification center behavior	Unread-count and mark-read chain tests	Verified
FR-17	Triage queue governance	Triage review UI and decision endpoint flow checks	Verified

Req ID	Requirement Theme	Validation Evidence	Status
FR-18	Analytics and heatmap	Admin analytics and map rendering walk-through	Verified
FR-19	CSV export	Export endpoint execution and output schema checks	Verified
FR-20	Public board	Public feed route and UI accessibility checks	Verified
FR-21	Health probes	Live, ready, model, and GPU endpoint smoke checks	Verified
FR-22	API version alias	/api/v1 integration checks in matrix tests	Verified